

The Coordination of Pricing and Scheduling Decisions

Zhi-Long Chen

Robert H. Smith School of Business, University of Maryland, College Park, Maryland 20742,
zchen@rhsmith.umd.edu

Nicholas G. Hall

Fisher College of Business, The Ohio State University, Columbus, Ohio 43210,
hall_33@fisher.osu.edu

This paper considers the coordination of pricing and scheduling decisions in a make-to-order environment. Following common industry practice, we assume knowledge of a deterministic demand function that is nonincreasing in price. We consider three alternative measures of scheduling cost: total work-in-process inventory cost of orders, total penalty for orders delivered late to customers, and total capacity usage. The objective is to maximize the total net profit, i.e., revenue less scheduling cost, resulting from the pricing and scheduling decisions. We develop computationally efficient optimal algorithms for solving the three pricing and scheduling problems. Because these problems are formally intractable, much faster algorithms are not possible. We develop a fully polynomial time approximation scheme for each problem. We also estimate the value of coordinating pricing and production scheduling decisions by comparing solutions delivered by (a) an uncoordinated approach where pricing and scheduling decisions are made independently, (b) a partially coordinated approach that uses only general information about scheduling that a marketing department typically knows, (c) a simple heuristic approach for solving the coordinated problem, and (d) our optimal algorithm for solving the coordinated problem. Our main managerial insight is that there is a significant benefit even if pricing and scheduling are only heuristically or partially coordinated. Moreover, heuristic and partial coordination are simple to achieve.

Key words: scheduling; pricing; optimal and approximate algorithms; value of coordinating decisions

History: Received: April 21, 2008; accepted: November 20, 2008. Published online in *Articles in Advance* April 8, 2009.

1. Introduction

In many manufacturing companies, pricing decisions are made by the marketing department. However, the production department must satisfy the demand that results from those pricing decisions. Although marketing departments often have general information about available production capacity, they typically do not have detailed knowledge about how the production requirements that result from various demand patterns will be scheduled. As a result, the effect of various pricing decisions on scheduling cost is not known to the marketing department when it makes those decisions. This paper examines the potential for improved profitability through the *coordination of pricing and production decisions*. The main contributions of this paper are the joint optimization of pricing and detailed scheduling decisions, and managerial insights

resulting from an evaluation of the benefits of coordination. One of the main insights is that there is a significant benefit even if pricing and scheduling decisions are only heuristically or partially coordinated.

The issue of coordinating pricing and production decisions has attracted significant research attention. Comprehensive literature surveys are provided by Eliashberg and Steinberg (1993) and Yano and Gilbert (2004). The literature on incentive design, although not closely related to our work, is important for this application. Porteus and Whang (1991) develop a system of linear transfer prices to align the decisions of marketing and production departments, based on principal-agent theory. The coordination scheme proposed by Lee and Whang (1999) is more complex and requires nonlinear payments. An alternative generalization is provided by Kouvelis and Lariviere (2000),

who construct internal markets in which the principal acts as a market maker. A recent overview of this research area is provided by Cachon (2003).

Optimization-based studies of this problem that consider *stochastic* demand include Thomas (1974) for production planning, Gallego and van Ryzin (1994) for the dynamic pricing of inventories, So and Song (1998) for joint pricing, delivery time guarantee, and capacity expansion decisions, Easton and Moodie (1999) for a make-to-order environment with uncertainty about bid outcomes, and Federgruen and Heching (1999, 2002) for joint pricing and inventory control.

However, we assume knowledge of a *deterministic* demand function for each product. There are two justifications for this approach. First, it is often difficult in practice to obtain an accurate distribution of demand scenarios, and hence a point estimate of expected demand is used deterministically for planning in many decision support systems. Second, a deterministic approach is also consistent with the literature on related problems (e.g., Thomas 1970; Kunreuther and Schrage 1973; Gilbert 1999, 2000; Deng and Yano 2006). Gilbert (2000) justifies this approach by pointing out (a) that for mature products the coefficient of variation in demand is often very low, and a deterministic model is reasonably accurate, (b) that in practice expected demand is often used deterministically at the planning stage, and (c) that a deterministic model is a first step toward understanding the more realistic and complicated environment with stochastic demand.

Within the deterministic literature, there are several studies that model the coordination of pricing and production decisions for a *single product*. Zhao and Wang (2002) consider an uncapacitated two-stage supply chain with perfect information, where a manufacturer outsources its retailing function to an independent distributor. Profits are measured by revenue less purchasing, ordering, and inventory holding costs. Uncoordinated solutions produce profits that are, on average, over 21% lower than those found by a coordinated decision-making approach. Deng and Yano (2006) consider the problem of choosing prices and production quantities for a single product, under capacity constraints, to maximize net profit for a manufacturer with deterministic demand that varies over time. They consider setup, production, and inventory holding costs, and provide extensive sensitivity

analyses and managerial insights. Geunes et al. (2006) consider a production problem that generalizes the classical economic lot sizing problem (Wagner and Whitin 1958). The objective is the maximization of net profit, which is measured as revenue less production and inventory holding costs. Under reasonable assumptions, they show that the problem can be solved in polynomial time. Ahn et al. (2007) study a manufacturing problem where the demand in a period is a function not only of the price in this period but also of the prices in previous periods. They identify structural properties of the problem and develop solution procedures.

There are also a few deterministic works on pricing and production planning decisions for *multiple products*. Several authors (Cheng 1990, Chen and Min 1994, Lee 1994) discuss problems in which all products are ordered with the same frequency, and storage capacity constraints are considered to various extents. Morgan et al. (2001) consider a similar problem in a make-to-stock environment. Dobson and Yano (2002) generalize this problem to include decisions about which products to offer as make-to-stock and which to offer as make-to-order. Gilbert (2000) and Charnsirisakskul et al. (2006) consider multiple product problems without the common production frequency assumption. Gilbert (2000) considers a model where several products share a common production capacity in a make-to-stock environment. His model assumes that a fixed price for each product is used across multiple periods with varying demand functions. The model maximizes net profit, which is measured by revenue less production and inventory holding costs. He provides extensive algorithmic development, a numerical example, and detailed sensitivity analysis results. Charnsirisakskul et al. (2006) develop a model for the coordination of pricing, order acceptance, scheduling, and lead time decisions. Orders incur a holding cost for early completion and a tardiness cost for late completion. The model is solved using an integer programming package for small instances and two heuristics for large instances.

All these existing models consider production decisions from an *aggregate planning* point of view. That is, the *detailed scheduling* of each individual order is not considered. Hence, the delivery of completed orders occurs at the end of a planning time period (e.g., one

week or one month), and completed orders are held in inventory to satisfy future demand. Furthermore, these models consider *aggregate planning costs*—setup, production, and finished product inventory holding costs. However, the consideration of detailed order-by-order scheduling decisions and costs is relevant in many practical environments. We provide several examples.

- For perishable products, such as materials used in the construction industry (Garcia and Lozano 2004, 2005; Geismar et al. 2008), each completed order is typically delivered individually and immediately after its completion. As a result, total cost and overall service-level measures depend on how each individual order is scheduled. Costs and service-level measures from aggregate planning models cannot be accurate, because they assume simultaneous delivery of all completed orders at the end of a planning time period and fail to consider individual order performance.

- In many make-to-order or assemble-to-order production systems for time-sensitive products, such as computers (Li et al. 2005, Stecke and Zhao 2007), there is no or little finished product inventory; hence, finished product inventory cost is negligible when considering the total cost. However, in such systems, a significant amount of components are kept as work-in-process (WIP) inventory, and the associated costs can be significant. Here an aggregate planning approach does not work, because the WIP inventory costs depend on when each order is scheduled.

- In a production system involving multiple stages such as a flowshop (Pinedo 2002), capacity usage at some stages cannot be estimated accurately without knowing a detailed order-by-order schedule because the amount of idle time in the schedule varies with the schedule. This is particularly problematic in pricing research because available capacity should be considered during the pricing decision. Therefore, an aggregate planning approach cannot efficiently plan capacity usage.

To summarize the above points, by considering scheduling costs at the individual order (or item) level, more complete and more accurate measurements of production costs and service levels are obtained. This in turn provides more accurate estimates of the

value of coordinated pricing and production decisions relative to uncoordinated decision making.

Motivated by the above discussion, we consider the *coordination of pricing and detailed scheduling decisions* in a make-to-order environment involving multiple products. Because detailed scheduling decisions typically involve a short planning horizon, we assume that a single price is used for each product over the entire planning horizon, and that the demand for a product realizes immediately after a price is chosen for the product at the beginning of the planning horizon. Similar assumptions are made in many existing papers (e.g., Kunreuther and Schrage 1973, Gilbert 2000, Charnsirisakskul et al. 2006), even for aggregate planning decisions that typically assume a longer planning horizon. We also assume that each incoming order requests exactly one item of some product and is delivered immediately upon its completion. This is common in make-to-order situations involving customized consumer products such as electronics and fashion items.

We consider three measures of scheduling cost. The first two measures are considered for a single-stage production system, as in almost all of the existing pricing and production literature with a finite production capacity. The first cost measure is the *total weighted completion time* of orders. This measure is commonly used as a proxy for the WIP inventory cost of the orders (Lane and Sidney 1993, Pinedo and Chao 1999). The second cost measure is the *total weight of late orders*. This measure is commonly used in the scheduling literature (Pinedo 2002) as the total lost revenue of the orders that are delivered late. The third measure of scheduling cost, which we consider for a two-stage production process, is the overall completion time, or *makespan*, of the orders. The makespan is commonly used as a proxy measure for capacity usage and for an overhead cost that accrues at a constant rate over time during production (Pinedo 2002).

We assume that due dates of orders are set by the manufacturer following rules that are commonly used in practice such as SLK or TWK (e.g., Cheng and Gupta 1989, Gordon et al. 2002). Using the SLK rule, the due date of an order is set to its processing time plus a common slack. Using the TWK rule, the due date of an order is set to a common multiplier times the processing time of the order. In either case,

different orders of the same product share a common due date.

This paper is organized as follows. In §2, we define the three problems to be studied. Section 3 presents optimal algorithms and intractability results for the maximization of net profit, using the above measures of scheduling cost. Section 4 contains a comparison of four approaches to solving the profit maximization problem: (a) a completely uncoordinated approach where the marketing department sets prices for products without considering the potential implications for scheduling costs, and the manufacturing department then schedules the incoming orders, (b) a partially coordinated heuristic that uses general information about production that is typically available to marketing departments, (c) a simple coordinated heuristic that uses detailed scheduling information, and (d) the optimal algorithm from §3. These four approaches are extensively tested against each other computationally. The results provide detailed managerial insights about the value of coordinating pricing and scheduling decisions. Section 5 contains a summary of our results and some suggestions for future research. Finally, the online companion contains a description of a fully polynomial time approximation scheme for each of the three problems discussed.

2. Preliminaries

In this section, we define the three problems to be studied. Let $N = \{1, \dots, n\}$ denote the set of n available products. For any product $j \in N$, let $q_{1j} > \dots > q_{m_j, j}$ denote the m_j allowable prices that can be set for product j . We assume knowledge of a deterministic, nonincreasing discrete demand function, $g_j(q_{ij})$. This function is specified by (price, demand) pairs $(q_{ij}, g_j(q_{ij}))$ for $i = 1, \dots, m_j$, $j = 1, \dots, n$. Here, $g_j(q_{ij})$ denotes the number of orders (or items) of product j that are demanded at price q_{ij} . We assume that q_{1j} is a sufficiently high price that $g_j(q_{1j}) = 0$. This is a reasonable assumption because, in many practical situations, setting the price high enough that there is no demand is an available option. We let $Q_j = \{q_{1j}, \dots, q_{m_j, j}\}$, $m_{\max} = \max_{1 \leq j \leq n} \{m_j\}$, and $M = \sum_{j=1}^n m_j$. We also let $\bar{g}_j = g_j(q_{m_j, j})$ denote the maximum possible demand for product j , and $\bar{g}_{\max} =$

$\max_{1 \leq j \leq n} \{\bar{g}_j\}$. It follows that the revenue for product j at price q_{ij} is $R_j(q_{ij}) = q_{ij}g_j(q_{ij})$.

Any incoming orders for products $1, \dots, n$ must be processed nonpreemptively on either a single-machine or a two-machine flowshop. In a single-machine environment, we let $p_j > 0$ denote the processing time of an order of product j . Alternatively, in a two-machine flowshop environment, we let $p_{1j} > 0$ and $p_{2j} > 0$ denote the processing time of an order for product j on machines M_1 and M_2 , respectively. Some problems that we consider also have a weight $w_j > 0$ or a due date $d_j > 0$ for each order of product j . We assume throughout that all p_j , d_j , and w_j are known integers. We further assume that all of the orders are available for processing at the start of the planning horizon.

Because price is a decision variable in our paper, and the demand function is deterministic, the decision maker effectively chooses the demand from a discrete set. Therefore, we make the simplifying assumption that the demand that is implied by the pricing decision for each product must be satisfied in full. This assumption is particularly realistic where the demand function has many discrete points, which enables the decision maker to choose the demand precisely. The results in this paper can easily be extended to the case where part or all of incoming demand can be rejected.

A solution σ to a problem consists of a price $x_j \in Q_j$ for each product $j \in N$ and a schedule π of all the incoming orders. Given a solution σ , we define the following variables:

$$R(\sigma) = \sum_{j \in N} R_j(x_j), \text{ the total revenue of} \\ \text{the incoming orders;}$$

$$C_{ij}(\sigma) = \text{the completion time of the } i\text{th order of} \\ \text{product } j, \text{ for } i = 1, \dots, g_j(x_j), j \in N;$$

$$U_{ij}(\sigma) = \begin{cases} 1, & \text{if } C_{ij}(\sigma) > d_j \\ 0, & \text{otherwise,} \end{cases} \\ \text{for } i = 1, \dots, g_j(x_j), j \in N.$$

Where the solution being used is clear from context, we simplify $R(\sigma)$, $C_{ij}(\sigma)$, and $U_{ij}(\sigma)$ to R , C_{ij} , and U_{ij} , respectively.

The standard classification scheme for scheduling problems (Graham et al. 1979) is $\alpha | \beta | \gamma$, where α indicates the scheduling environment, β describes

the order characteristics or restrictive requirements, and γ defines the objective function to be minimized. We consider single-machine problems where $\alpha = 1$, and two-machine flowshop problems where $\alpha = F2$. The problems considered here have the β field empty. We consider $\gamma \in \{\sum \sum w_j C_{ij} - R, \sum \sum w_j U_{ij} - R, C_{\max} - R\}$, where $\sum \sum w_j C_{ij} = \sum_{j=1}^n w_j \sum_{i=1}^{g_j(x_j)} C_{ij}$ is the total weighted completion time of incoming orders, $\sum \sum w_j U_{ij} = \sum_{j=1}^n w_j \sum_{i=1}^{g_j(x_j)} U_{ij}$ is the total weight of late orders, and $C_{\max} = \max_{1 \leq i \leq g_j(x_j), 1 \leq j \leq n} \{C_{ij}\}$ is the makespan (i.e., the overall schedule length). In each case, the objective is the maximization of net profit. We assume that the revenue function is normalized to match the time-based units of scheduling cost. As discussed in §1, the total weighted completion time represents the work-in-process inventory cost. Also, it is appropriate to use the total weight of late orders as the scheduling cost in situations where orders can be delivered late but with a reduction in the revenue. Finally, the makespan of a schedule represents overhead and capacity costs. Because setting the price high enough that there is no demand is always an option, the optimal value of γ is nonpositive. Let σ^* denote an optimal combination of price and schedule, i.e., one that maximizes the net profit.

Although our detailed analysis does not consider production cost, a linear production cost can be easily incorporated into our objective functions by redefining the price q_{ij} to be $q_{ij} - c_j$, where c_j is the unit production cost for processing an order of product j . The assumption of linearity for production costs is common in the literature (Gilbert 1999, 2000; Deng and Yano 2006; Charnsirisakskul et al. 2006).

An alternative approach to the problems we consider is to maximize total revenue, subject to a scheduling cost constraint that implies a reasonable service level. Most of the results in this paper can be modified to work for these alternative models.

In §3.1, we consider a *shortest weighted processing time* (SWPT) rule, in which the orders are sequenced by nonincreasing ratio of weight to processing time. In §3.2, we consider an *earliest due date* (EDD) rule, in which the orders are sequenced by nondecreasing due date. Finally, in §3.3 we consider the sequencing rule of Johnson (1954). Each of these sequencing rules enables us to find an optimal schedule for the problem considered in the respective section. A dynamic

programming algorithm uses the sequence to determine the other decisions that are necessary to define an optimal solution.

We use the following scheme to name the algorithms described in §3 and in the online companion. First, “C” denotes the total weighted completion time objective, “L” denotes the weighted number of late orders objective, and “M” denotes the makespan scheduling objective. Second, optimization algorithms have “OA” appended, and approximation schemes have “AS” appended.

3. Optimal Algorithms and Intractability

In this section, we study the three net profit maximization problems described in §2. In §3.1, we consider a single-machine environment, where the scheduling cost is determined by the total weighted completion time. In §3.2, we again consider a single-machine environment, where the scheduling cost is determined by the total weight of late orders. Finally, in §3.3, we consider a two-machine flowshop problem, where the scheduling cost is determined by the makespan. In each case, we provide an optimal dynamic programming algorithm and a related intractability result.

We note that our optimal algorithms are all based on an optimality property that, for the underlying scheduling problem without the pricing decisions, it is optimal to schedule the products in a particular prespecified sequence. The dynamic programs consider the products in this sequence and enumerate all possible prices for a product. This algorithmic framework is not limited to the three problems we consider here; it can also be applied to other problems that satisfy such an optimality property. For example, a similar optimality property holds for the single-machine maximum lateness problem without the pricing decisions and for the parallel-machine total completion time problem without the pricing decisions. Therefore, these problems with pricing decisions can also be solved using dynamic programming algorithms that are similar to those described here.

3.1. Completion Time

We present an optimal algorithm for maximizing the net profit, as determined by revenue less the total weighted completion time. It follows from Smith

(1956) that, for any instance of problem 1 $\| \sum \sum w_j C_{ij} - R$, there exists an optimal schedule in which the orders of each product are scheduled consecutively, and the products are scheduled in SWPT order.

ALGORITHM COA.

Input

Given $q_{ij}, g_j(q_{ij}), i = 1, \dots, m_j, j = 1, \dots, n; p_j, w_j, j = 1, \dots, n$.

Initialization

Index the products in SWPT order.

Value Function

$f_j(t)$ = maximum net profit from the orders of products $1, \dots, j$, given that after of product j is scheduled, the makespan of the schedule is t .

Boundary Condition

$f_0(0) = 0; f_j(t) = -\infty$, for $t < 0, j = 1, \dots, n$.

Optimal Solution Value

$\max_{0 \leq t \leq \sum_{j=1}^n \bar{g}_j p_j} \{f_n(t)\}$.

Recurrence Relation

$$f_j(t) = \max_{1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij}) - w_j g_j(q_{ij}) [t - p_j g_j(q_{ij})] - w_j p_j g_j(q_{ij}) [g_j(q_{ij}) + 1] / 2 + f_{j-1}(t - p_j g_j(q_{ij}))\}.$$

Algorithm COA considers each product in turn in SWPT sequence, and chooses the optimal number of orders of that product for the demand and the schedule. The second term in the maximization expression is the increment to total cost that results from starting $g_j(q_{ij})$ orders of product j at time $[t - p_j g_j(q_{ij})]$, rather than at time 0. The third term is the additional cost of scheduling those orders individually.

THEOREM 1. Algorithm COA finds an optimal schedule for problem 1 $\| \sum \sum w_j C_{ij} - R$ in $O(M \sum_{j=1}^n \bar{g}_j p_j)$ time.

PROOF. The indexing of the orders in the initialization step, and the single pass through the products in the recurrence relation, follow from Smith (1956). The optimality of Algorithm COA then follows from the fact that it compares the net profit from all possible state transitions. For the computation time, the value function $f_j(t)$ has $O(n)$ possible values for j and $O(\sum_{j=1}^n \bar{g}_j p_j)$ possible values for t . The recurrence relation is computed over $O(m_j)$ possible

values for q_{ij} . Therefore, the overall computation time of Algorithm COA is $O(M \sum_{j=1}^n \bar{g}_j p_j)$. $\square$

The following result shows that the pseudopolynomial time solution procedure provided in Algorithm COA is the best possible type of result, unless $P = NP$.

THEOREM 2. The recognition version of problem 1 $\| \sum \sum w_j C_{ij} - R$ is binary NP-complete, even when $p_1 = \dots = p_n = 1$ and $w_1 = \dots = w_n = 1$.

PROOF. See the online companion. $\square$

3.2. Late Orders

We present an optimal algorithm for maximizing the net profit, as determined by revenue less the total weight of late orders. It follows from Jackson (1955) that, for any instance of problem 1 $\| \sum \sum w_j U_{ij} - R$, there exists an optimal schedule in which the on-time orders of each product are scheduled consecutively, and the on-time orders are sequenced in EDD order.

ALGORITHM LOA.

Input

Given $q_{ij}, g_j(q_{ij}), i = 1, \dots, m_j, j = 1, \dots, n; p_j, w_j, d_j, j = 1, \dots, n$.

Initialization

Index the products in EDD order.

Value Function

$f_j(t)$ = maximum net profit from products $1, \dots, j$, given that after product j is scheduled, the makespan of the schedule of on-time orders is t .

Boundary Condition

$f_0(0) = 0; f_j(t) = -\infty$, for $t > d_j$ or $t < 0, j = 1, \dots, n$.

Optimal Solution Value

$\max_{0 \leq t \leq \sum_{j=1}^n \bar{g}_j p_j} \{f_n(t)\}$.

Recurrence Relation

$$f_j(t) = \max_{1 \leq i \leq m_j, 0 \leq X \leq g_j(q_{ij})} \{q_{ij} X + (q_{ij} - w_i)(g_j(q_{ij}) - X) + f_{j-1}(t - X p_j)\}, \text{ if } t \leq d_j.$$

Algorithm LOA considers each product in turn in EDD sequence, and chooses the optimal price and the optimal number of orders, X , to schedule on time. In the recurrence relation, the first term is the revenue from scheduling orders on time, and the second term is the revenue from scheduling orders late.

THEOREM 3. Algorithm LOA finds an optimal schedule for problem 1 $\parallel \sum \sum w_j U_{ij} - R$ in $O((\sum_{j=1}^n m_j \bar{g}_j) \cdot (\sum_{j=1}^n \bar{g}_j p_j))$ time.

PROOF. The proof of optimality is similar to that in Theorem 1. For the computation time, the value function $f_j(t)$ has $O(n)$ possible values for j and $O(\sum_{j=1}^n \bar{g}_j p_j)$ possible values for t . The recurrence relation is computed over $O(m_j)$ possible values for q_{ij} and $O(g_j(q_{ij}))$ possible values for X . Therefore, the overall computation time of Algorithm LOA is $O((\sum_{j=1}^n m_j \bar{g}_j)(\sum_{j=1}^n \bar{g}_j p_j))$. $\square$

The following result shows that the pseudopolynomial time algorithm provided in Algorithm LOA is the best possible type of result, unless $P = NP$.

THEOREM 4. The recognition version of problem 1 $\parallel \sum \sum w_j U_{ij} - R$ is binary NP-complete, even when $w_1 = \dots = w_n$ and $d_1 = \dots = d_n$.

PROOF. See the online companion. $\square$

3.3. Makespan

We present an optimal algorithm for maximizing the net profit, as determined by revenue less the two-machine flowshop makespan. It is easy to show by an interchange argument that, for any instance of problem F2 $\parallel C_{\max} - R$, there exists an optimal schedule in which the orders of each product are scheduled consecutively on both machines, and the products are scheduled in the sequence proposed by Johnson (1954). That is, the orders are first partitioned into two sets, $S_1 = \{j \mid p_{1j} \leq p_{2j}\}$ and $S_2 = \{j \mid p_{1j} > p_{2j}\}$. The orders of S_1 are scheduled first in nondecreasing p_{1j} sequence, followed by the orders of S_2 in nonincreasing p_{2j} sequence. The profile of a partial schedule is the difference between the completion time of the last order on machine M_2 and that on machine M_1 (Pinedo 2002).

ALGORITHM MOA.

Input

Given $q_{ij}, g_j(q_{ij}), i = 1, \dots, m_j, j = 1, \dots, n; p_{1j}, p_{2j}, j = 1, \dots, n$.

Initialization

Index the products in the sequence proposed by Johnson (1954).

Value Function

$f_j(\delta)$ = maximum net profit from products $1, \dots, j$, given that after product j is scheduled, the profile of the schedule is δ .

Boundary Condition

$f_0(0) = 0; f_j(\delta) = -\infty$, for $\delta < p_{2j}$ or $\delta > \sum_{i=1}^j p_{2i}$, $j = 1, \dots, n$.

Optimal Solution Value

$\max_{0 \leq \delta \leq \sum_{j=1}^n \bar{g}_j p_{2j}} \{f_n(\delta)\}$.

Recurrence Relations

(i) If $\delta > p_{2j}$,

$$f_j(\delta) = \begin{cases} \max_{1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij}) - p_{2j} g_j(q_{ij}) + f_{j-1}(\delta + (p_{1j} - p_{2j}) g_j(q_{ij}))\}, & \text{if } p_{1j} \leq p_{2j} \\ \max_{1 \leq i \leq m_j \mid g_j(q_{ij}) \leq \lfloor [\delta + g_j(q_{ij})(p_{1j} - p_{2j}) - p_{2j}] / (p_{1j} - p_{2j}) \rfloor} \{q_{ij} g_j(q_{ij}) - p_{2j} g_j(q_{ij}) + f_{j-1}(\delta + (p_{1j} - p_{2j}) g_j(q_{ij}))\}, & \text{if } p_{1j} > p_{2j}. \end{cases}$$

(ii) If $\delta = p_{2j}$,

$$f_j(p_{2j}) = \begin{cases} \max_{1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij}) - p_{2j} g_j(q_{ij}) + f_{j-1}(\delta + (p_{1j} - p_{2j}) g_j(q_{ij}))\}, & \text{if } p_{1j} \leq p_{2j} \\ \max_{0 \leq \delta' \leq \sum_{k=1}^{j-1} \bar{g}_k p_{2k}, 1 \leq i \leq m_j \mid g_j(q_{ij}) > \lfloor (\delta' - p_{2j}) / (p_{1j} - p_{2j}) \rfloor} \{q_{ij} g_j(q_{ij}) - p_{1j} g_j(q_{ij}) - p_{2j} + \delta' + f_{j-1}(\delta')\}, & \text{if } p_{1j} > p_{2j}. \end{cases}$$

Algorithm MOA considers each product in turn in the sequence proposed by Johnson (1954) and chooses the optimal price for that product. There are two cases for the profile δ . Consider the first case, where $\delta > p_{2j}$. If $p_{1j} \leq p_{2j}$, then no additional idle time is created on machine M_2 by the scheduling of $g_j(q_{ij})$ orders of product j . Alternatively, if $p_{1j} > p_{2j}$, then the profile is reduced by scheduling $g_j(q_{ij})$ orders of product j ; however, this case only occurs if $g_j(q_{ij})$ is small enough that no additional idle time is created on machine M_2 . Now consider the second case, where $\delta = p_{2j}$. If $p_{1j} \leq p_{2j}$, then the above argument for the same condition still applies. Alternatively, if $p_{1j} > p_{2j}$, then the new profile of p_{2j} can only be achieved if $g_j(q_{ij})$ is large enough that the completion time of the last order on machine M_1 is no earlier than the completion time of the second-last order on machine M_2 .

THEOREM 5. *Algorithm MOA finds an optimal schedule for problem $F2 \parallel C_{\max} - R$ in $O(M \sum_{j=1}^n \bar{g}_j p_{2j})$ time.*

PROOF. The proof of optimality is similar to that of Theorem 1. For the computation time, the value function $f_j(\delta)$ has $O(n)$ possible values for j and $O(\sum_{j=1}^n \bar{g}_j p_{2j})$ possible values for δ . When $\delta > p_{2j}$, both recurrence relations are computed over $O(m_j)$ possible values for q_{ij} . When $\delta = p_{2j}$, the first recurrence relation is computed over $O(m_j)$ possible values for q_{ij} . The second recurrence relation is computed over $O(m_j)$ possible values for q_{ij} and $O(\sum_{j=1}^n \bar{g}_j p_{2j})$ possible values for δ' ; however, the value of δ is unique for each j . Therefore, the overall computation time of Algorithm MOA is $O(M \sum_{j=1}^n \bar{g}_j p_{2j})$. $\square$

The following result shows that the pseudopolynomial time algorithm provided in Algorithm MOA is the best possible type of result, unless $P = NP$.

THEOREM 6. *The recognition version of problem $F2 \parallel C_{\max} - R$ is binary NP-complete.*

PROOF. See the online companion. $\square$

4. Heuristics and Managerial Insights

In this section, we estimate the value of coordinating pricing and production scheduling decisions by comparing solutions generated by four approaches with increasing levels of coordination and optimization. These four approaches are (a) an uncoordinated heuristic (H1) where pricing and scheduling decisions are made independently, (b) a partially coordinated heuristic (H2) that uses only basic information about scheduling that a marketing department typically knows, (c) a simple heuristic (H3) for solving the coordinated problem, and (d) the optimally coordinated approach given in §3 (i.e., Algorithms COA, LOA, and MOA). The three heuristics are described in §4.1. In §4.2, we describe our computational experiment, discuss its results, and provide several managerial insights.

4.1. Heuristics

The first two heuristics follow typical practices by marketing and manufacturing departments in many organizations. In the first heuristic, H1, the pricing decision is first made by maximizing the total revenue without considering its impact on scheduling costs, and then the scheduling decision is made

by minimizing the total scheduling cost, given the demand that is determined by the pricing decision. Because scheduling is not considered, the pricing decision can be made separately for each product.

HEURISTIC 1 (H1).

Step 1. For $j = 1, \dots, n$, choose price $q_{k_j, j}$ for product j , where $k_j = \arg \max_{1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij})\}$.

Step 2. Given $g_j(q_{k_j, j})$ orders of product j , for $j = 1, \dots, n$, schedule these orders optimally to minimize the total scheduling cost.

In Step 2 for problem $1 \parallel \sum \sum w_j C_{ij} - R$, it is optimal to schedule the orders in SWPT order (Smith 1956). Similarly for problem $F2 \parallel C_{\max} - R$, it is optimal to schedule the orders by Johnson's (1954) rule. However, the scheduling problem in Step 2 for problem $1 \parallel \sum \sum w_j U_{ij} - R$ is binary NP-hard (Karp 1972). Therefore, we use the pseudo-polynomial time dynamic program of Lawler and Moore (1969) to solve this problem.

In H2, the pricing decision is again made separately for each product, but the cost of scheduling the demand for that product is considered when the pricing decision is made. However, the contribution of that product to the scheduling cost of the other products, and the contribution of the other products to the scheduling cost of that product, are both ignored.

HEURISTIC 2 (H2).

Step 1. For $j = 1, \dots, n$, choose price $q_{k_j, j}$ for product j , where $k_j = \arg \max_{1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij}) - z_j(g_j(q_{ij}))\}$, and where $z_j(g_j(q_{ij}))$ is the cost of scheduling $g_j(q_{ij})$ orders of product j , starting from time 0.

Step 2. See Step 2 of H1.

In Step 1, the scheduling cost $z_j(g_j(q_{ij}))$ is calculated assuming that there are no other products on the machine(s). For problems $1 \parallel \sum \sum w_j C_{ij} - R$ and $F2 \parallel C_{\max} - R$, the scheduling cost $z_j(g_j(q_{ij}))$ can easily be calculated because all the orders of a product are identical, and the underlying scheduling problem can easily be solved for all the three problems. However, for problem $1 \parallel \sum \sum w_j U_{ij} - R$, we first schedule as many orders as possible on time, subject to the due date constraint, and then compute the cost of scheduling the remaining orders after the due date.

In contrast to H1 and H2, where there is either no coordination or only partial coordination, our algorithms COA, LOA, and MOA described in §3 jointly

optimize the pricing and scheduling decisions. However, it may be difficult to implement these complex algorithms in practice. Therefore, we propose as an alternative a simple heuristic, H3, which considers the pricing and scheduling decisions jointly. If the marketing and manufacturing departments fully coordinate their operations, knowledge of the production sequence is available to the marketing department and hence may be used in its pricing decisions. Thus, in H3, the products are considered in a sequence that is optimal with respect to the scheduling cost, and the heuristic pricing decision for each product is made based on both revenue and scheduling cost information.

HEURISTIC 3 (H3).

Step 0. Index the products in SWPT order, EDD order, or the order prescribed by Johnson's (1954) rule, for problem $1 \parallel \sum w_j C_{ij} - R$, $1 \parallel \sum w_j U_{ij} - R$, or $F2 \parallel C_{\max} - R$, respectively.

Step 1. For $j = 1, \dots, n$, choose price $q_{k_j, j}$ for product j , where $k_j = \arg \max_{1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij}) - y_j(g_j(q_{ij}))\}$, where $y_j(g_j(q_{ij}))$ is the cost of scheduling $g_j(q_{ij})$ orders of product j , after the $g_1(q_{k_1, 1})$ orders of product 1, $\dots$, $g_{j-1}(q_{k_{j-1}, j-1})$ orders of product $j-1$, which have been scheduled earlier.

Step 2. See Step 2 of H1.

The cost function $y_j(g_j(q_{ij}))$ in Step 1 of H3 for the three problems is computed similarly to $z_j(g_j(q_{ij}))$ in H2. However, whereas $z_j(g_j(q_{ij}))$ in H2 assumes that the orders start at time 0, $y_j(g_j(q_{ij}))$ in H3 considers the processing time of orders that have already been scheduled. Note that the decisions about q_{ij} made in this way are locally optimal in that the net profit of the current product j , but not of the whole schedule, is being maximized.

The performance of these heuristics is compared computationally in the next subsection. Our computational results show that for all three problems, on average, H3 performs better than H2, which performs better than H1. More specifically, we prove, below for problem $1 \parallel \sum w_j C_{ij} - R$, that performance of H3 dominates that of H2, which dominates that of H1 for every instance. However, for the other two problems, our computational results show that there exist problem instances for which H1 performs better than H2, and there exist problem instances for which H2 performs better than H3.

THEOREM 7. For any instance of problem $1 \parallel \sum w_j C_{ij} - R$, the profit generated by H3 is at least as large as the profit generated by H2, which is at least as large as the profit generated by H1.

PROOF. Let $k_{j1}, k_{j2}, k_{j3} \in \{1, \dots, m_j\}$ denote the price level chosen for product $j \in N$ by H1, H2, and H3, respectively. By comparing the ways in which these heuristics choose prices for the products, it is evident that $g_j(q_{k_{j1}, j}) \geq g_j(q_{k_{j2}, j}) \geq g_j(q_{k_{j3}, j})$, and hence $k_{j1} \leq k_{j2} \leq k_{j3}$, for $j \in N$. Let z_{ji} denote the cost of scheduling $g_j(q_{k_{ji}, j})$ orders of product j , starting from time 0, for $i = 1, 2, 3$ and $j \in N$. Let $R_{ji} = q_{k_{ji}, j} g_j(q_{k_{ji}, j})$, for $i = 1, 2, 3$. Let F_1 , F_2 , and F_3 denote the net profit of the solution generated by H1, H2, and H3, respectively. We have

$$F_i = \sum_{j \in N} R_{ji} - \sum_{j \in N} z_{ji} - \sum_{j \in N} \left[w_j g_j(q_{k_{ji}, j}) \sum_{v=1}^{j-1} p_v g_v(q_{k_{vi}, v}) \right],$$

for $i = 1, 2, 3$. (1)

From Step 1 of H2,

$$R_{j2} - z_{j2} \geq R_{j1} - z_{j1}, \quad \text{for } j \in N. \quad (2)$$

The fact that $g_j(q_{k_{j1}, j}) \geq g_j(q_{k_{j2}, j})$, for $j \in N$, implies that

$$w_j g_j(q_{k_{j1}, j}) \sum_{v=1}^{j-1} p_v g_v(q_{k_{v1}, v}) \geq w_j g_j(q_{k_{j2}, j}) \sum_{v=1}^{j-1} p_v g_v(q_{k_{v2}, v}),$$

for $j \in N$. (3)

Then, from (1), (2), and (3), we have $F_2 \geq F_1$.

Further, from Step 1 of H3,

$$R_{j3} - z_{j3} - w_j g_j(q_{k_{j3}, j}) \sum_{v=1}^{j-1} p_v g_v(q_{k_{v3}, v})$$

$$\geq R_{j2} - z_{j2} - w_j g_j(q_{k_{j2}, j}) \sum_{v=1}^{j-1} p_v g_v(q_{k_{v3}, v}),$$

for $j \in N$. (4)

The fact that $g_j(q_{k_{j2}, j}) \geq g_j(q_{k_{j3}, j})$, for $j \in N$, implies that

$$w_j g_j(q_{k_{j2}, j}) \sum_{v=1}^{j-1} p_v g_v(q_{k_{v2}, v}) \geq w_j g_j(q_{k_{j3}, j}) \sum_{v=1}^{j-1} p_v g_v(q_{k_{v3}, v}),$$

for $j \in N$. (5)

Then, from (1), (4), and (5), we have $F_3 \geq F_2$. $\square$

4.2. Computational Results and Insights

Our computational experiment addresses the following questions: (i) How much improvement in net profit can be achieved between the uncoordinated H1, the partially coordinated H2, the fully coordinated H3, and the optimally coordinated algorithm? (ii) How do these improvements vary with problem parameters? (iii) What other solution characteristics change as the level of coordination increases?

Our generation scheme follows the principles and properties for generating experimental data described in Hall and Posner (2001). In particular, we generate a variety of data to make fair comparisons between the performance of the three heuristics and the optimal algorithm for each problem. Also, the data generation scheme is easy to describe and implement.

To generate test problems, we use the linear demand function $g_j(q_{ij}) = \max\{0, \lfloor \alpha_j - \beta_j q_{ij} \rfloor\}$, where $\alpha_j, \beta_j > 0$. This demand function is commonly used in the literature (Gilbert 2000, Deng and Yano 2006). Our preliminary testing showed that the test results are mainly influenced by the following parameters: (i) number of products (n), (ii) number of allowable prices (m_j), (iii) demand at each price (α_j and β_j in the demand function), (iv) relative values of cost and revenue for each order (w_j compared to q_{ij} in problems $1 \parallel \sum \sum w_j C_{ij} - R$ and $1 \parallel \sum \sum w_j U_{ij} - R$, and p_{1j} and p_{2j} compared to q_{ij} in problem $F2 \parallel C_{\max} - R$), and (v) tightness of the due dates (d_j compared to the total processing time of the incoming orders). We therefore use different values or ranges for n , m_j , α_j , β_j , w_j , p_{1j} , p_{2j} , and d_j . However, we use just one range for q_{ij} and p_j , in accordance with the parsimony property (Hall and Posner 2001).

Our parameter settings are as follows:

(a) The number of products $n \in \{10, 50\}$, and the number of price levels m_j for each product j is an integer from $U[2, 6]$ or $U[4, 12]$.

(b) Prices of each product $q_{1j}, \dots, q_{m_j, j}$ are integers equally spaced within the interval $[x, y]$, where x and y are integers from $U[1,000, 3,000]$ and $U[8,000, 10,000]$, respectively.

(c) Parameters α_j and β_j used in the demand function are generated as $\alpha_j \sim U[20, 40]$ and $\beta_j \sim U[0.0015, 0.0025]$, or as $\alpha_j \sim U[35, 65]$ and $\beta_j \sim U[0.0035, 0.0065]$, respectively.

(d) For problems $1 \parallel \sum \sum w_j C_{ij} - R$ and $1 \parallel \sum \sum w_j U_{ij} - R$, the processing time p_j of each product is

an integer from $U[1, 10]$; for problem $F2 \parallel C_{\max} - R$, both p_{1j} and p_{2j} are integers from $U[500, 3,500]$ or $U[1,000, 7,000]$.

(e) For problem $1 \parallel \sum \sum w_j C_{ij} - R$, the weight w_j of each product is an integer from $U[1, 10]$ or $U[1, 20]$ when $n = 10$, and from $U[1, 3]$ or $U[1, 6]$ when $n = 50$. For problem $1 \parallel \sum \sum w_j U_{ij} - R$, w_j is an integer from $U[500, 5,000]$ or $U[1,000, 10,000]$.

(f) For problem $1 \parallel \sum \sum w_j U_{ij} - R$, the due date d_j of each product is an integer from $U[1, \dots, \gamma P_{\max}]$, where $P_{\max}$ is the total processing time of the incoming orders when the median price is used for each product, and $\gamma \in \{0.3, 0.7\}$.

These parameter ranges are designed to cover a wide variety of practical situations. The demand for a product varies from 0 up to 60 orders. The demand is much more sensitive to the price when $\beta_j \sim U[0.0035, 0.0065]$ than when $\beta_j \sim U[0.0015, 0.0025]$. For problem $1 \parallel \sum \sum w_j C_{ij} - R$, the ranges for the weight w_j of an order when $n = 10$ and $n = 50$ are specified such that in both cases the resulting “average” cost contributed by an order scheduled in the middle of an “average” schedule is either about 25% or about 50% of the average price of an order. Here we define an “average” schedule to be one where (i) half of the products have a nonzero demand, (ii) each product with a nonzero demand has a median number of orders (about 20), and (iii) each order has an average processing time (about 5). In this “average” schedule, the total processing time of the orders is $5 \times 20 \times 5 = 500$ when $n = 10$ and $25 \times 20 \times 5 = 2,500$ when $n = 50$. Therefore, the completion time of an order in the middle of the schedule is about 250 when $n = 10$ and 1,250 when $n = 50$, and its cost is about $250w_j$ or $1,250w_j$. Similarly, for problem $1 \parallel \sum \sum w_j U_{ij} - R$, the ranges of w_j are specified such that the “average” cost of a late order (which is w_j) is approximately equal to the “average” price of the order, respectively. Finally, for problem $F2 \parallel C_{\max} - R$, the ranges of p_{1j} and p_{2j} are specified such that the cost of an average order (which is between p_{2j} and $p_{1j} + p_{2j}$) is approximately equal to the “average” price of the order. The values of γ correspond to the cases where the due date d_j of a product j in problem $1 \parallel \sum \sum w_j U_{ij} - R$ is tight and loose.

Tables 1 through 3 show our computational results for the three problems. Each row summarizes the

Table 1 Comparison of Heuristic and Optimal Solutions for Problem 1 $\parallel \sum \sum w_j C_{ij} - R$

n	m_j	α_j, β_j	w_j	Profit gap %						Demand gap %			
				H1		H2		H3		H1	H2	H3	
				Mean	Stdev	Mean	Stdev	Mean	Stdev	Mean	Mean	Mean	
10	[2, 6]	[20, 40],	[1, 10]	1.53	2.34	1.07	1.94	0.41	1.05	4.51	3.82	2.17	
		[0.0015, 0.0025]	[1, 20]	12.92	15.96	10.97	14.91	3.46	4.11	19.39	18.02	9.53	
		[35, 65],	[1, 10]	11.26	9.77	7.15	7.00	1.94	2.21	20.45	16.53	7.64	
		[0.0035, 0.0065]	[1, 20]	52.01	43.54	33.31	32.04	6.00	5.33	45.22	36.56	13.67	
	[4, 12]	[20, 40],	[1, 10]	2.11	1.89	1.34	1.41	0.35	0.53	7.00	5.58	2.39	
		[0.0015, 0.0025]	[1, 20]	9.65	10.05	6.31	8.22	2.27	2.97	16.26	13.48	7.31	
		[35, 65],	[1, 10]	15.63	11.30	9.12	6.65	2.09	1.69	29.38	22.83	9.41	
		[0.0035, 0.0065]	[1, 20]	58.84	43.77	29.97	23.27	5.51	4.11	53.15	38.87	13.86	
	50	[2, 6]	[20, 40],	[1, 3]	4.49	1.85	4.22	1.74	1.74	1.07	11.25	11.01	7.56
			[0.0015, 0.0025]	[1, 6]	34.79	13.90	33.56	13.30	8.17	2.85	36.71	36.16	16.69
			[35, 65],	[1, 3]	27.32	8.52	24.81	7.56	5.36	2.02	36.38	34.91	15.46
			[0.0035, 0.0065]	[1, 6]	109.00	35.37	97.55	31.39	11.08	3.10	69.89	66.44	20.45
[4, 12]		[20, 40],	[1, 3]	4.59	1.31	4.19	1.22	0.91	0.38	10.31	9.84	4.20	
		[0.0015, 0.0025]	[1, 6]	23.85	8.63	22.04	8.41	6.51	2.59	31.24	30.22	15.58	
		[35, 65],	[1, 3]	33.95	8.56	30.66	7.93	4.74	1.30	46.08	43.88	15.80	
		[0.0035, 0.0065]	[1, 6]	115.15	33.81	98.81	29.73	10.54	2.79	79.48	74.08	21.51	
Overall means				32.32	15.66	25.94	12.30	4.44	2.38	32.29	28.89	11.45	

results over 100 randomly generated instances. The columns “Mean” and “Stdev” represent the mean and standard deviation of the results over the 100 random instances, respectively. The columns under “Profit gap” show the relative gap (expressed as a percentage) in net profit between the solutions generated by H1, H2, and H3 and the optimal coordinated solution, respectively, i.e., $(Z_{DP} - Z_{H1})/Z_{DP}$, $(Z_{DP} - Z_{H2})/Z_{DP}$, and $(Z_{DP} - Z_{H3})/Z_{DP}$, respectively, where Z_{DP} is the net profit of an optimal coordinated solution, and Z_{H1} , Z_{H2} , and Z_{H3} are the net profits of the solutions generated by H1, H2, and H3, respectively. The columns under “Demand gap” show the relative gap (expressed as a percentage) in total realized demand of all the products between the solutions of H1, H2, and H3 and the optimal coordinated solution, i.e., $(N_{H1} - N_{DP})/N_{DP}$, $(N_{H2} - N_{DP})/N_{DP}$, and $(N_{H3} - N_{DP})/N_{DP}$, respectively, where N_{DP} is the total number of orders scheduled in an optimal coordinated solution, and N_{H1} , N_{H2} , and N_{H3} are the total numbers of orders scheduled in the solution generated by H1, H2, and H3, respectively. Because each heuristic takes

less than 10 CPU seconds and each dynamic programming algorithm takes less than 100 CPU seconds to solve any test problem on a 1 GHz personal computer, we do not report computation times in detail.

We first discuss the overall performance of the three heuristics. The value of partial coordination in H2 compared to no coordination in H1 can be seen in reduced mean profit gaps, from 32.32% to 25.94% in problem 1 $\parallel \sum \sum w_j C_{ij} - R$, from 7.35% to 5.38% in problem 1 $\parallel \sum \sum w_j U_{ij} - R$, and from 15.96% to 4.09% in problem F2 $\parallel C_{\max} - R$. Further significant improvements are offered by full coordination in H3, which gives mean profit gaps of 4.44%, 2.02%, and 2.84% for the three problems, respectively. Moreover, for all three problems, the standard deviations of the profit gaps associated with H3 are significantly smaller than those associated with H2, which are smaller than those associated with H1. This means that the performance of H1 is more consistent than that of H2, which is more consistent than that of H3. Because H3 routinely delivers solutions that are close to optimal, we recommend it as a sim-

Table 2 Comparison of Heuristic and Optimal Solutions for Problem 1 $\parallel \sum \sum w_j U_{ij} - R$

n	m _j	α _j , β _j	w _j	γ	Profit gap %						Demand gap %		
					H1		H2		H3		H1	H2	H3
					Mean	Stdev	Mean	Stdev	Mean	Stdev	Mean	Mean	Mean
10	[2, 6]	[20, 40], [0.0015, 0.0025]	[500, 5,000]	0.3	0.81	1.00	0.50	0.69	0.11	0.33	2.93	2.08	0.05
				0.7	0.40	0.63	0.29	0.52	0.13	0.32	1.86	1.51	0.49
			[1,000, 10,000]	0.3	9.85	8.38	7.06	7.48	3.78	4.88	23.96	19.11	1.42
				0.7	2.82	3.03	2.03	2.77	2.37	3.41	8.46	6.83	0.39
		[35, 65], [0.0035, 0.0065]	[500, 5,000]	0.3	4.56	3.18	1.97	2.15	0.66	1.25	12.74	6.85	−1.82
				0.7	2.56	2.24	1.72	1.76	0.59	0.81	9.18	7.10	−0.51
			[1,000, 10,000]	0.3	28.11	17.99	14.98	12.76	4.76	5.39	44.67	29.77	0.36
				0.7	11.58	8.82	7.85	6.98	3.19	3.28	22.68	17.89	0.07
	[4, 12]	[20, 40], [0.0015, 0.0025]	[500, 5,000]	0.3	1.65	1.23	0.99	0.89	0.27	0.33	6.53	4.60	−0.08
				0.7	0.85	0.86	0.66	0.79	0.35	0.46	4.35	3.76	0.86
			[1,000, 10,000]	0.3	8.20	5.62	5.43	4.38	2.68	4.34	19.67	15.18	2.20
				0.7	3.13	2.74	2.43	2.45	1.24	1.60	8.93	7.60	2.85
		[35, 65], [0.0035, 0.0065]	[500, 5,000]	0.3	7.34	3.74	3.64	2.81	1.08	1.20	22.52	13.36	−2.53
				0.7	4.02	2.50	2.70	2.03	1.24	1.19	15.49	12.09	−0.71
			[1,000, 10,000]	0.3	33.71	18.27	18.19	12.94	4.69	4.61	51.90	33.88	−1.93
				0.7	15.17	9.34	10.60	7.89	3.53	2.88	28.88	23.10	1.19
50	[2, 6]	[20, 40], [0.0015, 0.0025]	[500, 5,000]	0.3	0.87	0.39	0.82	0.38	0.16	0.15	3.14	3.01	0.24
				0.7	0.33	0.19	0.32	0.19	0.20	0.14	2.09	2.04	0.87
			[1,000, 10,000]	0.3	7.96	2.86	7.45	2.74	4.63	2.38	20.22	19.23	−0.26
				0.7	1.57	0.89	1.45	0.82	2.31	1.40	5.47	5.10	−0.88
		[35, 65], [0.0035, 0.0065]	[500, 5,000]	0.3	3.93	1.32	3.49	1.25	0.56	0.37	12.84	11.79	−1.04
				0.7	1.86	0.82	1.71	0.83	0.80	0.42	8.63	8.21	0.56
			[1,000, 10,000]	0.3	22.19	6.01	20.06	5.67	5.70	2.56	39.05	36.21	−1.58
				0.7	7.52	2.82	6.91	2.86	4.15	1.91	17.97	17.00	0.10
	[4, 12]	[20, 40], [0.0015, 0.0025]	[500, 5,000]	0.3	1.37	0.46	1.27	0.44	0.27	0.15	5.95	5.62	0.16
				0.7	0.45	0.24	0.41	0.22	0.29	0.16	3.29	3.13	1.11
			[1,000, 10,000]	0.3	6.27	2.02	5.83	1.98	2.80	1.77	16.10	15.30	0.49
				0.7	1.59	0.76	1.43	0.73	1.44	1.05	5.91	5.62	1.71
		[35, 65], [0.0035, 0.0065]	[500, 5,000]	0.3	5.91	1.38	5.24	1.33	0.95	0.39	20.75	18.88	−2.61
				0.7	2.72	0.90	2.49	0.89	1.30	0.42	13.06	12.34	0.11
			[1,000, 10,000]	0.3	26.16	6.13	23.45	5.74	4.48	2.26	46.18	42.71	−1.86
				0.7	9.73	3.09	8.88	2.97	3.97	1.42	23.46	22.19	1.36
Overall means					7.35	3.75	5.38	3.07	2.02	1.66	16.53	13.53	0.03

pler alternative to the implementation of our optimal algorithms described in §3.

We also observe the following sensitivity analysis results regarding total net profit:

1. For all the three problems, when demand is more sensitive to price, i.e., the case with $\beta \sim [0.0035, 0.0065]$, all three heuristics show much larger profit gaps under every configuration of the other parameters. This is because high sensitivity of demand to price magnifies heuristic errors in price choices.

2. For problems $1 \parallel \sum \sum w_j C_{ij} - R$ and $1 \parallel \sum \sum w_j U_{ij} - R$, when the weight (w_j) is larger, all three heuristics show much larger profit gaps under every configuration of the other parameters. Similarly, for problem $F2 \parallel C_{\max} - R$, when processing times (p_{1j}, p_{2j}) are larger, all three heuristics show much larger profit gaps under every configuration of the other parameters. This is because H1 and H2 fail to consider fully the scheduling costs, which are more significant when w_j in the first two problems and p_{1j}, p_{2j} in the last problem are larger.

Table 3 Comparison of Heuristic and Optimal Solutions for Problem $F2 \parallel C_{\max} - R$

n	m_j	α_j, β_j	p_{1j}, p_{2j}	Profit gap %						Demand gap %		
				H1		H2		H3		H1	H2	H3
				Mean	Stdev	Mean	Stdev	Mean	Stdev	Mean	Mean	Mean
10	[2, 6]	[20, 40],	[500, 3,500]	1.05	1.07	0.07	0.20	0.06	0.20	3.07	−0.36	−0.06
		[0.0015, 0.0025]	[1,000, 7,000]	13.71	14.63	10.98	12.20	6.90	10.42	21.82	−15.73	−3.55
		[35, 65],	[500, 3,500]	4.90	2.70	0.34	0.57	0.26	0.55	14.47	−2.00	−0.34
		[0.0035, 0.0065]	[1,000, 7,000]	51.23	38.76	13.74	14.03	9.60	12.79	47.93	−19.48	−6.99
	[4, 12]	[20, 40],	[500, 3,500]	1.80	1.27	0.11	0.23	0.05	0.15	6.62	−0.91	−0.26
		[0.0015, 0.0025]	[1,000, 7,000]	11.19	7.95	2.35	5.13	1.14	3.75	12.10	−5.38	−1.01
		[35, 65],	[500, 3,500]	7.37	2.70	0.61	0.67	0.49	0.72	23.31	−3.64	−0.47
		[0.0035, 0.0065]	[1,000, 7,000]	55.92	29.72	6.51	7.13	3.55	5.87	53.20	−12.69	−3.56
50	[2, 6]	[20, 40],	[500, 3,500]	0.92	0.37	0.08	0.10	0.09	0.14	3.03	−0.53	−0.11
		[0.0015, 0.0025]	[1,000, 7,000]	8.71	3.03	8.83	3.88	6.17	5.17	16.38	−15.59	−3.88
		[35, 65],	[500, 3,500]	4.13	1.11	0.50	0.31	0.47	0.54	12.86	−2.85	−0.13
		[0.0035, 0.0065]	[1,000, 7,000]	34.16	8.91	11.30	4.31	9.44	6.26	40.57	−20.53	−7.12
	[4, 12]	[20, 40],	[500, 3,500]	1.63	0.42	0.17	0.12	0.18	0.22	6.73	−1.30	−0.20
		[0.0015, 0.0025]	[1000, 7,000]	8.99	2.22	2.67	2.10	1.35	1.89	12.20	−5.86	−1.12
		[35, 65],	[500, 3,500]	6.46	1.16	0.74	0.27	0.77	0.73	22.33	−4.70	−0.72
		[0.0035, 0.0065]	[1,000, 7,000]	43.18	8.93	6.40	2.64	4.90	3.94	51.69	−13.96	−3.37
Overall means				15.96	7.81	4.09	3.37	2.84	3.33	21.77	−7.84	−2.05

3. For problem $1 \parallel \sum \sum w_j U_{ij} - R$, when due dates are tighter, i.e., the case with $\gamma = 0.3$, H1 and H2 show much larger profit gaps under every configuration of the other parameters. This is because difficulty in meeting the due dates magnifies the cost of heuristic errors in scheduling choices. On the contrary, the performance of H3 does not vary much with the tightness of due dates and hence is more robust than H1 and H2.

4. For all three problems, the profit gap of H1 increases slightly with the range of m_j under most configurations of the other parameters. Our explanation for this is that when the number of products is small, having more price choices for each product significantly increases net profit for both the optimal algorithm and H1. However, because H1 does not choose effectively between different prices, the magnitude of profit increase for the optimal algorithm is more than that for H1, which results in a larger gap between the value of the optimal solution and the solution from H1 as the value of m_j increases. By contrast, the performance of H2 and H3 does not vary significantly with the range of m_j ; hence, they are more stable than H1.

We have the following observations about the total demand scheduled by the three heuristics and the optimal algorithm. Because it does not consider cost, H1 schedules substantially more orders than the optimal algorithm, giving average excess demands of 32.29%, 16.53%, and 21.77% for problems $1 \parallel \sum \sum w_j C_{ij} - R$, $1 \parallel \sum \sum w_j U_{ij} - R$, and $F2 \parallel C_{\max} - R$, respectively. Because H2 partially considers cost, it gives smaller excess demands of 28.89% and 13.53% compared to H1 for the first two problems, respectively. However, for problem $F2 \parallel C_{\max} - R$, H2 gives a demand shortfall of 7.84%, due to overestimation of the increase in flowshop makespan when an order is added. Because H3 evaluates cost more accurately, it gives a much lower excess demand (11.45%) than the other heuristics for problem $1 \parallel \sum \sum w_j C_{ij} - R$, and schedules very close to the same total demand as the optimal algorithm for the other two problems.

In addition, we have tested problem instances generated with another commonly used demand function, $g_j(q_{ij}) = \lfloor \alpha_j q_{ij}^{-\beta_j} \rfloor$, where $\alpha_j, \beta_j > 0$ are given constants. The results for this demand function are

similar to those in Tables 1, 2, and 3, and hence we do not report them in the paper.

Based on the above observations, we offer the following insights to managers:

1. In situations where demand is sensitive to price or where profit margins are relatively small, the coordination of pricing and scheduling decisions is particularly important. We provide algorithms for finding optimal coordinated solutions. However, for managers who require a solution that is easier to implement, we recommend using H3, which routinely provides near-optimal coordinated solutions for all the three problems studied here. The highly successful performance of H3 suggests that coordination is more important than overall optimization in joint pricing and scheduling decisions.

2. In situations where communication between marketing and production is poor and full coordination is therefore impossible, it is still valuable to use the partially coordinated H2 because it provides a significant improvement in average profit over the uncoordinated H1.

3. Although profit is in many cases the primary objective, an important secondary objective is market share, as measured by the percentage of realized demand enjoyed by the company. Here the heuristics usually perform well; however, H2 does not typically achieve a good market share for problem $F2 \parallel C_{\max} - R$.

4. For consistent success, it is best to align the incentives of the marketing and production departments. Therefore, performance incentives within the marketing department should be based on net profit, rather than revenue. Implementing such incentives requires communication of detailed schedules and their costs between the production and marketing departments.

5. Concluding Remarks

This paper considers three problems that require pricing and scheduling decisions. Although these problems are formally intractable, for each of them we describe computationally efficient algorithm that can be used to solve large instances to optimality. We also describe a simple heuristic that routinely provides close to optimal solutions in computational testing. This heuristic is compared with alternative

approaches that use either uncoordinated or only partially coordinated decision making for pricing and scheduling. We complete our study by providing several sensitivity analyses and managerial insights about the value of coordinating pricing and scheduling decisions, and about how to achieve such coordination.

The online companion describes fully polynomial time approximation schemes for the three problems studied. Such schemes find a solution with a predetermined degree of accuracy, in a computation time that increases only slowly with the degree of accuracy required. Fully polynomial time approximation schemes provide the strongest possible approximation results for intractable problems. Our approximation schemes are based on the optimal dynamic programming algorithms described in §§3.1, 3.2, and 3.3. The approximation schemes start with a classical state space trimming approach, but extend that approach to provide the desired results.

The main contributions of this paper are the joint optimization of pricing and detailed scheduling decisions, and various managerial insights resulting from an evaluation of the benefits. The main managerial insight is that there is a significant benefit to even partial or heuristic coordination, especially when demand is sensitive to price, profit margins are small, work-in-process holding costs or processing times are large, due dates are tightly constraining, or when there are many choices for prices. Moreover, partial and heuristic coordination are simple to achieve. Our work provides two new decision-making tools for coordinating pricing and scheduling decisions: an algorithm that requires some implementation effort but provides optimal solutions even for large problem instances, and a simple heuristic that routinely provides close to optimal solutions.

Several related topics are available for future research. First, there are other measures of scheduling cost, in addition to the three studied here, that can be considered within net profit. Second, the design and worst-case analysis of simple heuristics for intractable pricing and scheduling problems remains an open area. Finally, from a supply chain perspective, it would be valuable to coordinate additional decisions such as distribution in combination with pricing and scheduling. In conclusion, we hope that our work

will encourage the development of further research that supports the coordination of pricing and other operational decisions in manufacturing practice.

Electronic Companion

An electronic companion to this paper is available on the *Manufacturing & Service Operations Management* website (<http://msom.pubs.informs.org/ecompanion.html>).

Acknowledgments

This research is supported in part by National Science Foundation Grants DMI-0421637 to the first author and DMI-0421823 to the second author and by the Summer Fellowship Program of the Fisher College of Business at The Ohio State University to the second author.

References

- Ahn, H.-S., M. Gumus, P. Kaminsky. 2007. Pricing and manufacturing decisions when demand is a function of prices in multiple periods. *Oper. Res.* **55** 1039–1057.
- Cachon, G. P. 2003. Supply chain coordination with contracts. A. G. de Kok, S. C. Graves, eds. *Supply Chain Management: Design, Coordination and Operations*. Elsevier, Amsterdam, 229–340.
- Charnsirisakskul, K., P. M. Griffin, P. Keskinocak. 2006. Pricing and scheduling decisions with leadtime flexibility. *Eur. J. Oper. Res.* **171** 153–169.
- Chen, C.-K., K. J. Min. 1994. A multi-product EOQ model with pricing consideration—T.C.E. Cheng's model revisited. *Comput. Indust. Engrg.* **26** 787–794.
- Cheng, T. C. E. 1990. An EOQ model with pricing consideration. *Comput. Indust. Engrg.* **18** 529–534.
- Cheng, T. C. E., M. C. Gupta. 1989. Survey of scheduling research involving due date determination decisions. *Eur. J. Oper. Res.* **38** 156–166.
- Deng, S., C. A. Yano. 2006. Joint production and pricing decisions with setup costs and capacity considerations. *Management Sci.* **52** 741–756.
- Dobson, G., C. A. Yano. 2002. Product offering, pricing and make-to-stock/make-to-order decisions with shared capacity. *Production Oper. Management* **11** 293–312.
- Easton, F. R., D. R. Moodie. 1999. Pricing and lead time decisions for make-to-order firms with contingent orders. *Eur. J. Oper. Res.* **116** 305–318.
- Eliashberg, J., R. Steinberg. 1993. Marketing-production joint decision making. *Handbooks in Operations Research and Management Science*, Vol. 5. Elsevier Science Publishers, Amsterdam, 827–880.
- Federgruen, A., A. Heching. 1999. Combined pricing and inventory control under uncertainty. *Oper. Res.* **47** 454–475.
- Federgruen, A., A. Heching. 2002. Multilocation combined pricing and inventory control. *Manufacturing Service Oper. Management* **4** 275–296.
- Gallego, G., G. J. van Ryzin. 1994. Optimal dynamic pricing of inventories with stochastic demand over finite horizons. *Management Sci.* **40** 999–1020.
- Garcia, J. M., S. Lozano. 2004. Production and delivery scheduling problem with time windows. *Comput. Indust. Engrg.* **48** 733–742.
- Garcia, J. M., S. Lozano. 2005. Production and vehicle scheduling for ready-mix operations. *Comput. Indust. Engrg.* **46** 803–816.
- Geismar, H. N., G. Laporte, L. Lei, C. Sriskandarajah. 2008. The integrated production and transportation scheduling problem for a product with a short life span and non-instantaneous transportation time. *INFORMS J. Comput.* **20** 21–33.
- Geunes, J., H. E. Romeijn, K. Taaffe. 2006. Requirements planning with pricing and order selection flexibility. *Oper. Res.* **54** 394–401.
- Gilbert, S. M. 1999. Coordination of pricing and multi-period production for constant priced goods. *Eur. J. Oper. Res.* **114** 330–337.
- Gilbert, S. M. 2000. Coordination of pricing and multiple-period production across multiple constant priced goods. *Management Sci.* **46** 1602–1616.
- Gordon, V. S., J.-M. Proth, C. Chu. 2002. Due date assignment and scheduling: SLK, TWK and other due date assignment models. *Production Planning Control* **13** 117–132.
- Graham, R. L., E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan. 1979. Optimization and approximation in deterministic machine scheduling: A survey. *Ann. Discrete Math.* **5** 287–326.
- Hall, N. G., M. E. Posner. 2001. Generating experimental data for computational testing with machine scheduling applications. *Oper. Res.* **49** 854–865.
- Jackson, J. R. 1955. Scheduling a production line to minimize maximum tardiness. Research Report 43, Management Science Research Project, University of California, Los Angeles.
- Johnson, S. M. 1954. Optimal two and three-stage production schedules with setup times included. *Naval Res. Logist. Quart.* **1** 61–67.
- Karp, R. M. 1972. Reducibility among combinatorial problems. R. E. Miller, J. W. Thatcher, eds. *Complexity of Computer Computations*. Plenum Press, New York, 85–103.
- Kouvelis, P., M. A. Lariviere. 2000. Decentralizing cross-functional decisions: Coordination through internal markets. *Management Sci.* **46** 1049–1058.
- Kunreuther, H., L. Schrage. 1973. Joint pricing and inventory decisions for constant priced items. *Management Sci.* **19** 732–738.
- Lane, D. E., J. B. Sidney. 1993. Batching and scheduling in FMS hubs: Flow time considerations. *Oper. Res.* **41** 1091–1103.
- Lawler, E. L., J. M. Moore. 1969. A functional equation and its application to resource allocation and sequencing problems. *Management Sci.* **16** 77–84.
- Lee, W. J. 1994. Optimal order quantities and prices with storage space and inventory investment limitations. *Comput. Indust. Engrg.* **26** 481–488.
- Lee, H. L., S. Whang. 1999. Decentralized multi-echelon inventory control systems: Incentives and information. *Management Sci.* **45** 633–640.
- Li, K. P., V. K. Ganesan, A. I. Sivakumar. 2005. Synchronized scheduling of assembly and multi-destination air-transportation in a consumer electronics supply chain. *Internat. J. Production Res.* **43** 2671–2685.
- Morgan, L. O., R. L. Daniels, P. Kouvelis. 2001. Marketing/manufacturing tradeoffs in product line management. *IIE Trans.* **33** 949–962.

- Pinedo, M. 2002. *Scheduling: Theory, Algorithms and Systems*, 2nd ed. Prentice Hall, Upper Saddle River, NJ.
- Pinedo, M., X. Chao. 1999. *Operations Scheduling with Applications in Manufacturing and Services*. Irwin McGraw-Hill, New York.
- Porteus, E. L., S. Whang. 1991. On manufacturing/marketing incentives. *Management Sci.* **37** 1166–1181.
- Smith, W. E. 1956. Various optimizers for single stage production. *Naval Res. Logist. Quart.* **3** 59–66.
- So, K. C., J.-S. Song. 1998. Price, delivery time guarantees and capacity selection. *Eur. J. Oper. Res.* **111** 28–49.
- Stecke, K. E., X. Zhao. 2007. Production and transportation integration for a make-to-order manufacturing company with a commit-to-delivery business mode. *Manufacturing Service Oper. Management* **9** 206–224.
- Thomas, L. J. 1970. Price-production decisions with deterministic demand. *Management Sci.* **16** 747–750.
- Thomas, L. J. 1974. Price and production decisions with random demand. *Oper. Res.* **22** 513–518.
- Wagner, H. M., T. Whitin. 1958. Dynamic version of the economic lot size model. *Management Sci.* **5** 89–96.
- Yano, C. A., S. M. Gilbert. 2004. Coordinated pricing and production/procurement decisions: A review. A. Chakravarty, J. Eliashberg, eds. *Managing Business Interfaces: Marketing, Engineering and Manufacturing Perspectives*. Kluwer Academic Publishers, Norwell, MA.
- Zhao, W., Y. Wang. 2002. Coordination of joint pricing-production decisions in a supply chain. *IIE Trans.* **34** 701–715.

Online Appendix
for Paper “The Coordination of Pricing and Scheduling Decisions”
by Zhi-Long Chen and Nicholas G. Hall

Part A1: Proofs of Theorems 2, 4, and 6

Proof of Theorem 2. By reduction from the following problem, which is known to be binary *NP*-complete.

Partition (Garey and Johnson, 1979): given t elements with integer sizes $a_1, \dots, a_t$, where $\sum_{i=1}^t a_i = 2A$, does there exist a partition S_1, S_2 of the index set $\{1, \dots, t\}$ such that $\sum_{i \in S_1} a_i = \sum_{i \in S_2} a_i = A$?

Consider the following instance of the recognition version of problem 1|| $\sum \sum w_j C_{ij} - R$:

$$\begin{aligned} n &= t, \\ p_j &= 1, & j = 1, \dots, n \\ w_j &= 1, & j = 1, \dots, n \\ m_j &= 2, & j = 1, \dots, n \\ q_{1j} &= 2A, & j = 1, \dots, n \\ q_{2j} &= A + 1/2, & j = 1, \dots, n \\ g_j(q_{1j}) &= 0, & j = 1, \dots, n \\ g_j(q_{2j}) &= a_j, & j = 1, \dots, n \\ Z &= A^2/2, \end{aligned}$$

where Z denotes the threshold net profit.

We prove that there exists a feasible schedule for this instance of problem 1|| $\sum \sum w_j C_{ij} - R$ with $R - \sum \sum w_j C_{ij} \geq Z$ if and only if there exists a solution to Partition.

($\Rightarrow$) Assume that there exists a solution S_1, S_2 to Partition. For each product $j \in S_1$, set the price equal to q_{2j} , and schedule a_j incoming orders consecutively. For the other products, set the price equal to q_{1j} , hence there are no orders to schedule. Therefore, the net profit is

$$\begin{aligned} R - \sum \sum w_j C_{ij} &= (A + 1/2) \sum_{j \in S_1} a_j - \sum_{j \in S_1} [a_j(a_j + 1)/2 + a_j \sum_{i \in S_1, i < j} a_i] \\ &= A^2 + A/2 - \sum_{j \in S_1} a_j/2 - [\sum_{j \in S_1} a_j]^2/2 \\ &= Z. \end{aligned}$$

($\Leftarrow$) Let S denote the set of products that are priced at $A + 1/2$. Thus, there are a_j incoming orders of each product $j \in S$. Then, $R = (A + 1/2) \sum_{j \in S} a_j$. Since $p_j = 1$ and $w_j = 1$ for $j \in S$, any sequence of the incoming orders yields the same total weighted completion time. Therefore, we assume that the orders are scheduled in the order of their product indices. This implies that

$\sum \sum w_j C_{ij} = \sum_{j \in S} [a_j(a_j + 1)/2 + a_j \sum_{i \in S, i < j} a_i]$. Therefore, the net profit is

$$\begin{aligned}
R - \sum \sum w_j C_{ij} &= (A + 1/2) \sum_{j \in S} a_j - \sum_{j \in S} [a_j(a_j + 1)/2 + a_j \sum_{i \in S, i < j} a_i] \\
&= (A + 1/2) \sum_{j \in S} a_j - \sum_{j \in S} a_j/2 - [\sum_{j \in S} a_j]^2/2 \\
&= (\sum_{j \in S} a_j)(2A - \sum_{j \in S} a_j)/2.
\end{aligned} \tag{1}$$

Now, (1) achieves its maximum over all possible sets S when $\sum_{j \in S} a_j = 2A - \sum_{j \in S} a_j$, where $R - \sum \sum w_j C_{ij} = A^2/2 = Z$. Therefore, if $R - \sum \sum w_j C_{ij} \geq Z$, then $(S, N \setminus S)$ is a solution to Partition. $\square$

Proof of Theorem 4. Since problem 1|| $\sum \sum w_j U_{ij} - R$ is a generalization of the classical scheduling problem 1|| $\sum w_j U_j$, it follows from Karp (1972) that the recognition version of the former problem is binary *NP*-complete. We now prove that, even in the special case where $w_1 = \dots = w_n$ and $d_1 = \dots = d_n$, this result still holds. We use a reduction from Partition, as described in the proof of Theorem 2. Consider the following instance of the recognition version of problem 1|| $\sum \sum w_j U_{ij} - R$, where $w_1 = \dots = w_n$ and $d_1 = \dots = d_n$:

$$\begin{array}{lll}
n & = & t, \\
p_j & = & a_j, \quad j = 1, \dots, n \\
w_j & = & 2A, \quad j = 1, \dots, n \\
d_j & = & A, \quad j = 1, \dots, n \\
m_j & = & 2, \quad j = 1, \dots, n \\
q_{1j} & = & 2A, \quad j = 1, \dots, n \\
q_{2j} & = & a_j, \quad j = 1, \dots, n \\
g_j(q_{1j}) & = & 0, \quad j = 1, \dots, n \\
g_j(q_{2j}) & = & 1, \quad j = 1, \dots, n \\
Z & = & A,
\end{array}$$

where Z denotes the threshold net profit.

We prove that there exists a feasible schedule for this instance of problem 1|| $\sum \sum w_j U_{ij} - R$ with $R - \sum \sum w_j U_{ij} \geq Z$ if and only if there exists a solution to Partition.

($\Rightarrow$) Assume that there exists a solution S_1, S_2 to Partition. For each product $j \in S_1$, set the price equal to q_{2j} , which generates one order. For the other products, set the price equal to q_{1j} , which generates no orders. Schedule all the orders in arbitrary order. Since the total processing time of the orders is equal to $\sum_{j \in S_1} a_j = A = d_k$, for $k = 1, \dots, n$, no order is late and hence $\sum \sum w_j U_{ij} = 0$. Therefore, the net profit is $\sum_{j \in S_1} q_{2j} = \sum_{j \in S_1} a_j = A = Z$.

($\Leftarrow$) Let S denote the set of products with a price q_{2j} . Thus, there is one order of each product $j \in S$, and there are no orders of any other product. Then, $R = \sum_{j \in S} a_j$. Since $w_j - q_{2j} > Z$, for $j = 1, \dots, n$, no order is scheduled late. It follows that (i) $\sum_{j \in S} p_j \leq d_k$ for $k \in S$, which implies $\sum_{j \in S} a_j \leq A$; and (ii) $R \geq Z$, which implies $\sum_{j \in S} a_j \geq A$. Therefore, $\sum_{j \in S} a_j = A$ and $(S, N \setminus S)$ is a solution to Partition. $\square$

Proof of Theorem 6. By reduction from Partition, as described in the proof of Theorem 2. We consider the following instance of the recognition version of problem $F2||C_{\max} - R$:

$$\begin{aligned}
n &= t + 1 \\
p_{1j} &= 0, & j = 1, \dots, n-1 \\
p_{1n} &= A \\
p_{2j} &= a_j, & j = 1, \dots, n-1 \\
p_{2n} &= 1 \\
m_j &= 2, & j = 1, \dots, n \\
q_{1j} &= A, & j = 1, \dots, n-1 \\
g_j(q_{1j}) &= 0, & j = 1, \dots, n-1 \\
q_{1n} &= 2A^2 \\
g_n(q_{1n}) &= 0 \\
q_{2j} &= a_j/2, & j = 1, \dots, n-1 \\
g_j(q_{2j}) &= 1, & j = 1, \dots, n-1 \\
q_{2n} &= A^2 \\
g_n(q_{2n}) &= 1 \\
Z &= A^2 - A/2 - 1,
\end{aligned}$$

where Z denotes the threshold net profit, or $R - C_{\max}$.

We prove that there exists a feasible schedule for this instance of problem $F2||C_{\max} - R$ with $R - C_{\max} \geq Z$ if and only if there exists a solution to Partition.

($\Rightarrow$) Assume that there exists a solution S_1, S_2 to Partition. Price each product $j \in S_1 \cup \{n\}$ at q_{2j} , thereby creating a demand of one unit for that product. Price each product $j \in S_2$ at q_{1j} , thereby creating no demand. Schedule the order of product n in the interval $[0, A]$ on machine M_1 and the interval $[A, A + 1]$ on machine M_2 . Schedule one order of each product in S_1 in the interval $[0, A]$ on machine M_2 . The revenue R from scheduling the order of product n and the orders of S_1 is $A^2 + A/2$. Also, by construction, $C_{\max} = A + 1$. Therefore, the net profit is $R - C_{\max} = (A^2 + A/2) - (A + 1) = A^2 - A/2 - 1 = Z$.

($\Leftarrow$) Since $\sum_{j=1}^{n-1} q_{2j} < Z$, any solution to the pricing and scheduling problem with $R - C_{\max} \geq Z$ must schedule an order of product n . Since $C_n \geq A + 1$ in any schedule that includes an order of product n , the net profit from all other orders must be at least $Z - [A^2 - (A + 1)] = A/2$. Let S de-

note the set of other orders which are scheduled, and thus priced at q_{2j} . Since $p_{2j} = 2q_{2j}$ for $j \in S$, such orders can only be scheduled if they do not increase $C_{\max}$, i.e. they must be scheduled in the available idle time in $[0, A]$ on machine M_2 . It follows that $\sum_{j \in S} a_j \leq A$. However, if $\sum_{j \in S} p_{2j} < A$, then the additional net revenue earned from the orders of S is $\sum_{j \in S} q_{2j} = \sum_{j \in S} p_{2j}/2 < A/2$, hence $R - C_{\max} < Z$. Therefore, we must have $\sum_{j \in S} p_{2j} = A$, which implies that $(S, \{1, \dots, n\} \setminus S)$ is a solution to Partition. $\square$

Part A2: Approximation Schemes

In this part, we provide an FPTAS for each of the three pricing and scheduling problems discussed in Section 3. An FPTAS is a family of algorithms $\{A_\epsilon\}$ such that, for any given $\epsilon > 0$, A_ϵ delivers a solution that is within a relative error ϵ from optimality and has a running time that is polynomial in both the data and $1/\epsilon$. An FPTAS is the strongest possible approximation result for an intractable problem, unless $P = NP$. In the following we first provide an outline of our method for designing approximation schemes in Section A2.1, followed by our discussions in Sections A2.2, A2.3, and A2.4 how this method can be used to provide an FPTAS for problems $1||\sum\sum w_j C_{ij} - R$, $1||\sum\sum w_j U_{ij} - R$ and $F2||C_{\max} - R$, respectively.

A2.1 Algorithm design

We consider a dynamic programming algorithm that is similar to Algorithms COA, LOA and MOA in Sections 3.1, 3.2 and 3.3, respectively. However, we include the value function as a state. Our approximation schemes start with a state space trimming approach that can be used to design approximation schemes for classical scheduling problems without pricing decisions (Schuurman and Woeginger, 2006). However, in a classical scheduling problem the number of orders is given, whereas in our problems the number of orders is controlled by pricing decisions. Consequently, we develop techniques that extend the existing approach. In problem $1||\sum\sum w_j C_{ij} - R$, our approach requires partitioning the range of possible net profit values into intervals of equal width Δ , as defined below. However, in problems $1||\sum\sum w_j U_{ij} - R$ and $F2||C_{\max} - R$, it is necessary to use intervals of geometrically increasing width $\Delta, \Delta^2, \dots$, as also defined below. Within each interval, only one partial schedule is retained, and the others are discarded. This enables the dynamic program to run in a polynomial number of steps, while controlling the loss of accuracy by adjusting the value of Δ . Since the input information is the same as for the corresponding optimal algorithm, with the

addition of ϵ , we omit this step.

A2.2 Completion time

We begin our analysis of problem $1||\sum\sum w_j C_{ij} - R$ with a preliminary result.

Lemma 1 *In problem $1||\sum\sum w_j C_{ij} - R$, we have $F_0 \leq R(\sigma^*) - \sum\sum w_j C_{ij}(\sigma^*) \leq nF_0$, where $F_0 = \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij}g_j(q_{ij}) - w_j p_j g_j(q_{ij})[g_j(q_{ij}) + 1]/2\}$.*

Proof. F_0 is a lower bound on optimal net profit which is found by considering only one product. Since this value is calculated as the maximum over all products, an upper bound on optimal net profit is found by taking n times this value, and ignoring the interactive cost terms between products. $\square$

We now describe our fully polynomial time approximation scheme for problem $1||\sum\sum w_j C_{ij} - R$. This scheme is based on a dynamic programming algorithm with states (j, t, v) , indicating that products $1, \dots, j$ have been considered, and the schedule has makespan t and net profit v . Given two partial schedules (j, t', v') and (j, t'', v'') , where the values of v' and v'' are similar, we eliminate the second partial schedule if $t' \leq t''$, and the first partial schedule otherwise. For a value of x such that $i\Delta \leq x < (i+1)\Delta$, we define the label $\Delta(x) = \Delta i$.

Algorithm CAS $_\epsilon$

Initialization

Index the products in SWPT order.

Define $F_0 = \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij}g_j(q_{ij}) - w_j p_j g_j(q_{ij})[g_j(q_{ij}) + 1]/2\}$. Set $\Delta = \epsilon F_0/n$.

State Variables

(j, t, v) corresponds to a partial schedule that has considered products $1, \dots, j$, where the schedule has makespan t and net profit v .

Initial State

$(0, 0, 0)$.

Trial State Generation

For each state (j, t, v) , generate m_{j+1} trial states $(j+1, t', v')$ by pricing product $j+1$ at $q_{i,j+1}$ and then scheduling the resulting demand of $g_{j+1}(q_{i,j+1})$ orders, for $i = 1, \dots, m_{j+1}$.

Trial State Labelling

For each trial state $(j + 1, t', v')$, attach the label $(j + 1, \Delta(v'))$.

Trial State Elimination

For each pair of trial states $(j + 1, t', v')$ and $(j + 1, t'', v'')$ with identical labels, eliminate the second state if $t' \leq t''$, and eliminate the first state otherwise.

Termination Test

If $j + 1 < n$, then set $j = j + 1$ and return to the Trial State Generation step. Otherwise, select a state $(n, \tilde{t}, \tilde{v})$ for which $\tilde{v}$ is largest, and backtrack to find the corresponding schedule $\tilde{\sigma}_\epsilon$.

At the Initialization step, the value of Δ is set such that $n\Delta/\epsilon = F_0$, where from Lemma 1 F_0 is a lower bound on optimal net profit. The Trial State Generation step chooses the price $q_{i,j+1}$, where $1 \leq i \leq m_{j+1}$, for product $j + 1$ that generates a demand $g_{j+1}(q_{i,j+1})$ for that product, and then schedules those orders. The Trial State Labelling step labels each trial state, based on the number of orders considered and the net profit value. The Trial State Elimination step compares the profile of all trial states with the same label, keeps only one with the smallest makespan, and discards the others.

Theorem 1 *The family of algorithms $\{CAS_\epsilon\}$, for $\epsilon > 0$, is an FPTAS for problem*

$1|| \sum \sum w_j C_{ij} - R$, *with $O(n^3 m_{\max}/\epsilon)$ running time.*

Proof. We start by analyzing the cost of the solution delivered by Algorithm CAS_ϵ . The proof is by induction on the number of products that have been considered for scheduling. Our induction hypothesis is that, given any state (j, t, v) that is obtained under the exact dynamic program, Algorithm CAS_ϵ generates a state $(j, \tilde{t}, \tilde{v})$, where $\tilde{t} \leq t$ and $\tilde{v} \geq v - j\Delta$. The hypothesis clearly holds for $j = 0$. Suppose that the hypothesis holds for $j = 0, 1, \dots, k$. Let (k, t, v) define a state in the exact dynamic program.

We assume that the exact dynamic program prices product $k + 1$ at $q_{i,k+1}$, for some $1 \leq i \leq m_{k+1}$, and therefore that $g_{k+1}(q_{i,k+1})$ orders of product $k + 1$ are scheduled. We assume that, in the exact dynamic program, this yields a state $(k + 1, t', v')$. From the induction hypothesis, we have a state $(k, \tilde{t}, \tilde{v})$ in the approximate dynamic program where $\tilde{t} \leq t$ and $\tilde{v} \geq v - k\Delta$. The trial state, S , that is generated from $(k, \tilde{t}, \tilde{v})$ by pricing product $k + 1$ at $q_{i,k+1}$ and scheduling $g_{k+1}(q_{i,k+1})$ orders of product $k + 1$ in Algorithm CAS_ϵ is denoted by $(k + 1, \tilde{t}', \tilde{v}')$. We prove the following two claims

about this state.

- (i) $\tilde{t}' \leq t'$. From the induction hypothesis, $\tilde{t} \leq t$. Moreover, the same orders are added in the approximate and exact dynamic programs, therefore the increase in makespan is the same, i.e. $\tilde{t}' - \tilde{t} = t' - t$. Combining these inequalities establishes the claim.
- (ii) $\tilde{v}' \geq v' - (k+1)\Delta$. From the induction hypothesis, $\tilde{v} \geq v - k\Delta$. Now, $R_{k+1} = q_{i,k+1}g_{k+1}(q_{i,k+1})$ in both the exact and approximate dynamic programs. Since state $(k+1, t', v')$ is generated from state (k, t, v) , we have

$$v' = v + R_{k+1} - tg_{k+1}(q_{i,k+1})w_{k+1} - g_{k+1}(q_{i,k+1})[g_{k+1}(q_{i,k+1}) + 1]p_{k+1}w_{k+1}/2.$$

Similarly, since state $(k+1, \tilde{t}', \tilde{v}')$ is generated from state $(k, \tilde{t}, \tilde{v})$, we have

$$\tilde{v}' = \tilde{v} + R_{k+1} - \tilde{t}g_{k+1}(q_{i,k+1})w_{k+1} - g_{k+1}(q_{i,k+1})[g_{k+1}(q_{i,k+1}) + 1]p_{k+1}w_{k+1}/2.$$

These relations, together with the induction hypothesis, imply that $\tilde{v}' \geq v' - k\delta > v' - (k+1)\delta$.

It follows from claims (i) and (ii) that if trial state S exists in Algorithm CAS_ϵ , then the induction argument is complete.

However, trial state S may be eliminated by another trial state $S' = (k+1, \tau_1, \tau_2)$, where $\tau_1 \leq \tilde{t}'$ and $\tau_2 \geq \tilde{v}' - \Delta$, where the last bound holds because S and S' have the same label. Then, we have for S' that (i) $\tau_1 \leq \tilde{t}' \leq t'$ and (ii) $\tau_2 \geq \tilde{v}' - \Delta \geq v' - (k+1)\Delta$. Thus, the induction hypothesis holds.

We now show that Algorithm CAS_ϵ delivers a solution with the required performance guarantee. Let the state (n, t, v^*) correspond to an optimal solution with value v^* . Then, the above induction argument shows that Algorithm CAS_ϵ generates a solution $\tilde{\sigma}$ and a corresponding final state $(n, \tilde{t}, \tilde{v})$ with value $\tilde{v}$, where $\tilde{v} \geq v^* - n\Delta$. Therefore, $[(R(\sigma^*) - \sum \sum w_j C_{ij}(\sigma^*)) - (R(\tilde{\sigma}) - \sum \sum w_j C_{ij}(\tilde{\sigma}))]/[R(\sigma^*) - \sum \sum w_j C_{ij}(\sigma^*)] \leq n\Delta/[R(\sigma^*) - \sum \sum w_j C_{ij}(\sigma^*)] \leq \epsilon$, where the last inequality follows from the specification of Δ in the Initialization step and from Lemma 1.

Finally, we analyze the time complexity of Algorithm CAS_ϵ . We first evaluate the number of different labels that are used in the Trial State Labelling step. There are $O(n)$ values of $j+1$. From the specification of Δ in the Initialization step and from Lemma 1, there are $O(n^2/\epsilon)$ possible labels for $\Delta(v')$. Thus, the total number of labels is $O(n^3/\epsilon)$. Each state generates at most $O(m_{\max})$ trial states. It follows that the overall time complexity of Algorithm CAS_ϵ is $O(n^3 m_{\max}/\epsilon)$. $\square$

A2.3. Late orders

We describe an FPTAS for problem $1||\sum\sum w_j U_{ij} - R$. This scheme is based on a dynamic programming algorithm with states (j, t, v) , where the schedule has makespan t for the on-time orders and value v . Also, trial state elimination is based on makespan values, as in Algorithm CAS_ϵ . We partition the relevant part of the state space into intervals, where the ends of the intervals are placed at geometrically increasing points. We define

$$\Delta = 1 + \epsilon/2n, \quad (2)$$

which implies the inequality

$$\Delta^n \leq 1 + \epsilon, \quad 0 \leq \epsilon \leq 2. \quad (3)$$

For a value of x such that $\Delta^i \leq x < \Delta^{i+1}$, we define the label $\Delta(x) = \Delta^i$.

However, problem $1||\sum\sum w_j U_{ij} - R$ also requires a choice of how many orders of product j to schedule on-time, rather than late. This choice can be any number $x \in \{0, \dots, g_j(q_{ij})\}$. We now provide a procedure that restricts this choice to a polynomial number of options. The restricted set of choices is denoted by Y .

Procedure LimitChoice

Input

$\bar{g}_j, j = 1, \dots, n$.

Compute Restricted Choice Set

Let $\Delta = 1 + \epsilon/2n$.

Let $a = \max\{k \mid \Delta^k \leq \bar{g}_{\max}\}$.

Let $Y = \{\cup_{k=0}^{a+1} (\lfloor \Delta^k \rfloor \cup \lceil \Delta^k \rceil)\} \cup \{0\}$.

Output

Y , the restricted set of choices for the number of orders to schedule on-time.

For any integer $0 \leq x \leq \bar{g}_{\max}$, define $\lfloor x \rfloor_\Delta = \max\{z \in Y \mid z \leq x\}$. The following preliminary result establishes some properties of Y and $\lfloor x \rfloor_\Delta$ that are used below.

Lemma 2 $|Y| = O(n \log \bar{g}_{\max}/\epsilon)$, and $x/\Delta \leq \lfloor x \rfloor_\Delta \leq x$ for any integer $0 \leq x \leq \bar{g}_{\max}$.

Proof. Clearly, $a \leq \log_\Delta \bar{g}_{\max} = (\log \bar{g}_{\max})/(\log \Delta)$. Since $\log x \geq (x-1)/x$ for all $x > 1$, we have $\log \Delta \geq \epsilon/(2n + \epsilon)$. Therefore, $a \leq (\log \bar{g}_{\max})(1 + 2n/\epsilon)$. Since in Procedure LimitChoice, $|Y| \leq 2a + 5$, it follows that $|Y| = O((\log \bar{g}_{\max})(1 + 2n/\epsilon)) = O(n \log \bar{g}_{\max}/\epsilon)$.

To prove the second part of the lemma, we first observe that $\lfloor x \rfloor_\Delta \leq x$, from the definition of $\lfloor x \rfloor_\Delta$. If $x = 0$, then it is trivial to see that $\lfloor x \rfloor_\Delta \geq x/\Delta$. Alternatively, if $1 \leq x \leq \bar{g}_{\max}$, then there exists some $i \in \{0, \dots, a\}$ such that $\Delta^i \leq x \leq \Delta^{i+1}$, which, together with the fact that x is an integer, implies that $x \geq \lceil \Delta^i \rceil$. Since $\lceil \Delta^i \rceil \in Y$, the definition of $\lfloor x \rfloor_\Delta$ and the fact that $x \geq \lceil \Delta^i \rceil$ imply that $\lfloor x \rfloor_\Delta \geq \lceil \Delta^i \rceil \geq \Delta^i = \Delta^{i+1}/\Delta \geq x/\Delta$. $\square$

We are now ready to describe our FPTAS for problem 1|| $\sum \sum w_j U_{ij} - R$.

Algorithm LAS $_\epsilon$

Initialization

Index the products in EDD order.

Call Procedure LimitChoice.

Define $X_{ij} = \{\lfloor x \rfloor_\Delta \mid x \in \{0, \dots, g_j(q_{ij})\}\}$, $i = 1, \dots, m_j$, $j = 1, \dots, n$.

State Variables

(j, t, v) corresponds to a partial schedule that has considered products $1, \dots, j$, where the total processing time of the on-time orders scheduled is t and the net profit is v .

Initial State

$(0, 0, 0)$.

Trial State Generation

For each state (j, t, v) , for $i = 1, \dots, m_{j+1}$, generate $m_{j+1}|X_{i,j+1}|$ trial states $(j+1, t', v')$ by pricing product $j+1$ at $q_{i,j+1}$ and then scheduling x orders on-time and $[g_{j+1}(q_{i,j+1}) - x]$ orders late, for $x \in X_{i,j+1}$, respectively. Discard all infeasible trial states $(j+1, t', v')$ with $t' > d_{j+1}$.

Trial State Labelling

For each trial state $(j+1, t', v')$, attach the label $(j+1, \Delta(v'))$.

Trial State Elimination

For each pair of trial states $(j+1, t', v')$ and $(j+1, t'', v'')$ with identical labels, eliminate the second state if $t' \leq t''$, and eliminate the first state otherwise.

Termination Test

If $j+1 < n$, then set $j = j+1$ and return to the Trial State Generation step. Otherwise, select a state $(n, \tilde{t}, \tilde{v})$ for which $\tilde{v}$ is largest, and backtrack to find the corresponding schedule $\tilde{\sigma}_\epsilon$.

The overall structure of Algorithm LAS $_\epsilon$ is similar to that of Algorithm CAS $_\epsilon$. However, the

Trial State Generation step also implements the choice of how many of the orders to schedule on-time, over the restricted set of choices in X_{ij} , where the remainder are scheduled late.

Theorem 2 *The family of algorithms $\{LAS_\epsilon\}$, for $\epsilon > 0$, is an FPTAS for problem*

$1 || \sum \sum w_j U_{ij} - R$, with $O(n^3 m_{\max} \log \bar{g}_{\max} \log(n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij})\}) / \epsilon^2)$ running time.

Proof. We start by analyzing the cost of the solution delivered by Algorithm LAS_ϵ . Our induction hypothesis is that, given any state (j, t, v) that is obtained under the exact dynamic program, Algorithm LAS_ϵ generates a state $(j, \tilde{t}, \tilde{v})$, where $\tilde{t} \leq t$ and $\tilde{v} \geq v / \Delta^j$. The hypothesis clearly holds for $j = 0$. Suppose that the hypothesis holds for $j = 0, 1, \dots, k$. Let (k, t, v) define a state in the exact dynamic program.

We assume that the exact dynamic program prices product $k+1$ at $q_{i,k+1}$, for some $1 \leq i \leq m_{k+1}$, and therefore that $g_{k+1}(q_{i,k+1})$ orders of product $k+1$ are scheduled, of which x are scheduled on time. We assume that, in the exact dynamic program, this yields a state $(k+1, t', v')$. From the induction hypothesis, we have a state $(k, \tilde{t}, \tilde{v})$ in the approximate dynamic program where $\tilde{t} \leq t$ and $\tilde{v} \geq v / \Delta^k$. The trial state, S , that is generated from $(k, \tilde{t}, \tilde{v})$ by pricing product $k+1$ at $q_{i,k+1}$ and scheduling $g_{k+1}(q_{i,k+1})$ orders of product $k+1$, including $\lfloor x \rfloor_\Delta$ which are scheduled on time in Algorithm LAS_ϵ , is denoted by $(k+1, \tilde{t}', \tilde{v}')$. We prove the following two claims about this state.

(i) $\tilde{t}' \leq t'$. From the induction hypothesis, $\tilde{t} \leq t$. Moreover, the increase in the total processing time of on-time orders when orders of product $k+1$ are scheduled in the approximate dynamic program is $\tilde{t}' - \tilde{t} = p_{k+1} \lfloor x \rfloor_\Delta \leq p_{k+1} x = t' - t$, where the inequality follows from Lemma 2. Combining these inequalities establishes the claim.

(ii) $\tilde{v}' \geq v' / \Delta^{k+1}$. Now,

$$\begin{aligned} \tilde{v}' &= \tilde{v} + g_{k+1}(q_{i,k+1})(q_{i,k+1} - w_{k+1}) + w_{k+1} \lfloor x \rfloor_\Delta \\ &\geq v / \Delta^k + g_{k+1}(q_{i,k+1})(q_{i,k+1} - w_{k+1}) + w_{k+1} x / \Delta \\ &\geq [v + g_{k+1}(q_{i,k+1})(q_{i,k+1} - w_{k+1}) + w_{k+1} x] / \Delta^k \\ &= v' / \Delta^k \geq v' / \Delta^{k+1}, \end{aligned}$$

where the first inequality follows from the induction hypothesis and from Lemma 2. It follows from claims (i) and (ii) that if trial state S exists in Algorithm LAS_ϵ , then the induction argument is complete.

However, trial state S may be eliminated by another trial state $S' = (k+1, \tau_1, \tau_2)$, where $\tau_1 \leq \tilde{t}'$

and $\tau_2 \geq \tilde{v}'/\Delta$, where the last bound holds because S and S' have the same label. Then, we have for S' that (i) $\tau_1 \leq \tilde{t}' \leq t'$ and (ii) $\tau_2 \geq \tilde{v}'/\Delta \geq v'/\Delta^{k+1}$. Thus, the induction hypothesis holds.

We now show that Algorithm LAS_ϵ delivers a solution with the required performance guarantee. Let the state (n, t, v^*) correspond to an optimal solution with value v^* . Then, the above induction argument shows that Algorithm LAS_ϵ generates a solution $\tilde{\sigma}$ and a corresponding final state $(n, \tilde{t}, \tilde{v})$ with value $\tilde{v}$, where $\tilde{v} \geq v^*/\Delta^n$. Therefore, $[(R(\sigma^*) - \sum \sum w_j U_{ij}(\sigma^*)) - (R(\tilde{\sigma}) - \sum \sum w_j U_{ij}(\tilde{\sigma}))]/[R(\sigma^*) - \sum \sum w_j U_{ij}(\sigma^*)] \leq 1 - 1/\Delta^n \leq 1 - 1/(1 + \epsilon) \leq \epsilon$, where the second last inequality follows from (3).

Finally, we analyze the time complexity of Algorithm LAS_ϵ . We first evaluate the number of different labels that are used in the Trial State Labelling step. There are $O(n)$ values of $j + 1$. Since an upper bound on optimal net profit is given by $n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij})\}$, the number of possible values for the label $\Delta(v')$ is no more than $\lceil \log_\Delta(n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij})\}) \rceil = \lceil \log(n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij})\}) / \log \Delta \rceil \leq \lceil (1 + 2n/\epsilon) \log(n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij})\}) \rceil$, where the last inequality follows from (2) and the inequality $\log x \geq (x - 1)/x$ for all $x > 1$. Thus, the total number of labels is $O(n^2 \log(n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij})\})/\epsilon)$. From Lemma 2, each state generates at most $O(m_{\max} n \log \bar{g}_{\max}/\epsilon)$ trial states. Thus, the overall time complexity of Algorithm LAS_ϵ is $O(n^3 m_{\max} \log \bar{g}_{\max} \log(n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij})\})/\epsilon^2)$. $\square$

A2.4. Makespan

Our fully polynomial time approximation scheme for problem $F2||C_{\max} - R$ is based on a dynamic programming algorithm with states (j, h, v) , indicating that products $1, \dots, j$ have been considered, and the schedule has profile h and net profit v . Given two partial schedules for the orders of products $1, \dots, j$, with similar net profit values, we eliminate the partial schedule with smaller profile. For a value of x such that $\Delta^i \leq x < \Delta^{i+1}$, we define the label $\Delta(x) = \Delta^i$.

Algorithm MAS_ϵ

Initialization

Index the products in the sequence proposed by Johnson (1954).

Set $\Delta = 1 + \epsilon/2n$.

State Variables

(j, h, v) corresponds to a partial schedule that has considered products $1, \dots, j$, where the schedule

has profile h and net profit v .

Initial State

$(0, 0, 0)$.

Trial State Generation

For each state (j, h, v) , generate m_{j+1} trial states $(j + 1, h', v')$ by pricing product $j + 1$ at $q_{i,j+1}$ and then scheduling the resulting demand of $g_{j+1}(q_{i,j+1})$ orders, for $i = 1, \dots, m_{j+1}$.

Trial State Labelling

For each trial state $(j + 1, h', v')$, attach the label $(j + 1, \Delta(v'))$.

Trial State Elimination

For each pair of trial states $(j + 1, h', v')$ and $(j + 1, h'', v'')$ with identical labels, eliminate the second state if $h' \geq h''$, and eliminate the first state otherwise.

Termination Test

If $j + 1 < n$, then set $j = j + 1$ and return to the Trial State Generation step. Otherwise, select a state $(n, \tilde{h}, \tilde{v})$ for which $\tilde{v}$ is largest, and backtrack to find the corresponding schedule $\tilde{\sigma}_\epsilon$.

Algorithm MAS_ϵ is similar to Algorithm LAS_ϵ except that fewer trial states are generated in MAS_ϵ .

Theorem 3 *The family of algorithms $\{\text{MAS}_\epsilon\}$, for $\epsilon > 0$, is an FPTAS for problem $F2||C_{\max} - R$, with $O(n^2 m_{\max} \log(n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij} g_j(q_{ij})\})/\epsilon)$ running time.*

Proof. We start by analyzing the cost of the solution delivered by Algorithm MAS_ϵ . Our proof uses induction on j . The induction hypothesis is that, given any state (j, h, v) that is obtained under the exact dynamic program, Algorithm MAS_ϵ generates a state $(j, \tilde{h}, \tilde{v})$, where $\tilde{h} \geq h$ and $\tilde{v} \geq v/\Delta^j$. The hypothesis clearly holds for $j = 0$. Suppose that the hypothesis holds for $j = 0, 1, \dots, k$. Let (k, h, v) define a state in the exact dynamic program.

We assume that the exact dynamic program prices product $k + 1$ at $q_{i,k+1}$, for some $1 \leq i \leq m_{k+1}$, and therefore that $g_{k+1}(q_{i,k+1})$ orders of product $k + 1$ are scheduled. We assume that, in the exact dynamic program, this yields a state $(k + 1, h', v')$. From the induction hypothesis, we have a state $(k, \tilde{h}, \tilde{v})$ in the approximate dynamic program, where $\tilde{h} \geq h$ and $\tilde{v} \geq v/\Delta^k$. The trial state, S , that is generated from $(k, \tilde{h}, \tilde{v})$ by pricing product $k + 1$ at $q_{i,k+1}$ and scheduling $g_{k+1}(q_{i,k+1})$ orders of product $k + 1$ in Algorithm MAS_ϵ is denoted by $(k + 1, \tilde{h}', \tilde{v}')$. We prove the following

two claims about this state.

(i) $\tilde{h}' \geq h'$. From the induction hypothesis, $\tilde{h} \geq h$. The state $(k+1, \tilde{h}', \tilde{v}')$ in the approximate dynamic program is generated from $(k, \tilde{h}, \tilde{v})$ by adding the same set of orders as those that are added when $(k+1, h', v')$ is generated from (k, h, v) in the exact dynamic program. The fact that $\tilde{h} \geq h$ then implies that $\tilde{h}' \geq h'$.

(ii) $\tilde{v}' \geq v'/\Delta^{k+1}$. From the induction hypothesis, $\tilde{v} \geq v/\Delta^k$. Now, $R_{k+1} = q_{i,k+1}g_{k+1}(q_{i,k+1})$ in both the exact and approximate dynamic programs. By the same argument as in the proof of claim (i), it follows that the increase in makespan that results from the scheduling of product $k+1$ is no larger in the approximate dynamic program than in the exact dynamic program. Combining these facts, we have $\tilde{v}' - \tilde{v} \geq v' - v \geq v'/\Delta^k - v/\Delta^k$. This implies that $\tilde{v}' \geq \tilde{v} + v'/\Delta^k - v/\Delta^k \geq v'/\Delta^k \geq v'/\Delta^{k+1}$, where the second inequality follows from the induction hypothesis.

It follows from claims (i) and (ii) that if trial state S exists in Algorithm MAS_ϵ , then the induction argument is complete.

However, trial state S may be eliminated by another trial state $S' = (k+1, \tau_1, \tau_2)$, where $\tau_1 \geq \tilde{h}'$, and $\tau_2 \geq \tilde{v}'/\Delta$, and where the second bound holds because S and S' have the same label. Then, we have for S' that (i) $\tau_1 \geq \tilde{h}' \geq h'$, and (ii) $\tau_2 \geq \tilde{v}'/\Delta \geq v'/\Delta^{k+1}$. Thus, the presence of trial state S' in Algorithm MAS_ϵ establishes that the induction hypothesis holds.

We now show that Algorithm MAS_ϵ delivers a solution with the required performance guarantee. Let the state (n, h, v) correspond to an optimal solution with value v^* . Then, the above induction argument shows that Algorithm MAS_ϵ generates a solution $\tilde{\sigma}$ and a corresponding final state $(n, \tilde{h}, \tilde{v})$ with value $\tilde{v}$, where $\tilde{v} \geq v^*/\Delta^n$. Therefore, $[(R(\sigma^*) - C_{\max}(\sigma^*)) - (R(\tilde{\sigma}) - C_{\max}(\tilde{\sigma}))]/[R(\sigma^*) - C_{\max}(\sigma^*)] \leq 1 - 1/\Delta^n \leq 1 - 1/(1 + \epsilon) \leq \epsilon$, where the second last inequality follows from (3).

Finally, we analyze the time complexity of Algorithm MAS_ϵ . We first evaluate the number of different labels that are used in the Trial State Labelling step. There are $O(n)$ values of $j+1$. Since an upper bound on optimal net profit is given by $n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij}g_j(q_{ij})\}$, by the same argument as in the proof of Theorem 2, the number of possible values for the label $\Delta(v')$ is $O(n \log(n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij}g_j(q_{ij})\})/\epsilon)$. Thus, the total number of labels is $O(n^2 \log(n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij}g_j(q_{ij})\})/\epsilon)$. Each state generates at most $O(m_{\max})$ trial states. Thus, the overall time complexity of Algorithm MAS_ϵ is $O(n^2 m_{\max} \log(n \max_{1 \leq j \leq n, 1 \leq i \leq m_j} \{q_{ij}g_j(q_{ij})\})/\epsilon)$. $\square$

References

- Garey, M.R., D.S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, San Francisco.
- Johnson, S.M. 1954. Optimal two and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly* **1** 61–67.
- Karp, R.M. 1972. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, ed. R.E. Miller, J.W. Thatcher, Plenum Press, New York.
- Schuurman, P., G.J. Woeginger. 2006. Approximation schemes - a tutorial. To appear in *Lectures on Scheduling*, ed. R.H. Möhring, C.N. Potts, A.S. Schulz, G.J. Woeginger, L.A. Wolsey.